

# Complete DSA Course in Python — Beginner to FAANG Level

## A Professional Course Handbook

**Author:** Senior DSA Instructor & Technical Content Creator  
**Language:** Python 3.x  
**Level:** Absolute Beginner → FAANG Interview Ready  
**Philosophy:** Learn by understanding, not memorizing.

### Course Modules

| #  | Module                         | File                         | Difficulty       |
|----|--------------------------------|------------------------------|------------------|
| 1  | Complexity Analysis & Big-O    | 01_complexity_analysis.md    | ★ Beginner       |
| 2  | Arrays & Strings               | 02_arrays_and_strings.md     | ★ Beginner       |
| 3  | Linked Lists                   | 03_linked_lists.md           | ★ ★ Intermediate |
| 4  | Stacks & Queues                | 04_stacks_and_queues.md      | ★ ★ Intermediate |
| 5  | Hashing & HashMaps             | 05_hashing.md                | ★ ★ Intermediate |
| 6  | Recursion & Backtracking       | 06_recursion_backtracking.md | ★ ★ Intermediate |
| 7  | Sorting & Searching            | 07_sorting_searching.md      | ★ ★ Intermediate |
| 8  | Trees (Binary Trees, BST, AVL) | 08_trees.md                  | ★ ★ ★ Advanced   |
| 9  | Heaps & Priority Queues        | 09_heaps.md                  | ★ ★ ★ Advanced   |
| 10 | Graphs                         | 10_graphs.md                 | ★ ★ ★ Advanced   |
| 11 | Dynamic Programming            | 11_dynamic_programming.md    | ★ ★ ★ ★ Expert   |
| 12 | Greedy Algorithms              | 12_greedy_algorithms.md      | ★ ★ ★ Advanced   |
| 13 | Tries & Advanced Structures    | 13_tries_advanced.md         | ★ ★ ★ ★ Expert   |
| 14 | FAANG Interview Cheatsheet     | 14_interview_cheatsheet.md   | 🔥 All Levels     |

### Learning Roadmap

Week 1-2: Complexity Analysis → Arrays & Strings  
Week 3-4: Linked Lists → Stacks & Queues

Week 5-6: Hashing → Recursion & Backtracking  
Week 7-8: Sorting & Searching → Trees  
Week 9-10: Heaps → Graphs  
Week 11-13: Dynamic Programming → Greedy  
Week 14-16: Tries, Advanced Topics → Mock Interviews

---



## How to Use This Course

1. **Read** the explanation and analogy first
  2. **Code** the implementation yourself before looking at the solution
  3. **Analyze** the time & space complexity
  4. **Solve** practice problems in order (easy → hard)
  5. **Review** the revision summary before moving on
  6. **Revisit** common mistakes and interview tips
- 



## PDF Conversion Instructions

### Option 1: VS Code Extension (Recommended)

1. Install “**Markdown PDF**” extension in VS Code
2. Open any .md file → Ctrl+Shift+P → “Markdown PDF: Export (pdf)”

### Option 2: Pandoc (Command Line)

```
pip install pandoc
pandoc 00_index.md 01_complexity_analysis.md 02_arrays_and_strings.md ... -o
DSA_Course_Complete.pdf
```

### Option 3: Online Tool

1. Go to [markdowntopdf.com](https://markdowntopdf.com)
2. Paste the combined markdown content
3. Download as PDF

### Option 4: Python Script

```
# pip install markdown pdfkit
import markdown, pdfkit

files = [f"{i:02d}_*.md" for i in range(15)]
```

```
combined = ""
for f in sorted(glob.glob("*.md")):
    with open(f) as file:
        combined += file.read() + "\n\n---\n\n"

html = markdown.markdown(combined, extensions=['fenced_code', 'tables'])
pdfkit.from_string(html, "DSA_Course_Complete.pdf")
```

---

*Let's begin. Open Module 01 to start your DSA journey!*

---

# Module 1: Complexity Analysis & Big-O Notation

## 1.1 What is Complexity Analysis?

**Definition:** Complexity analysis measures how the **time** (speed) and **space** (memory) requirements of an algorithm grow as the input size increases.

### Real-Life Analogy

Imagine you're looking for a friend's name in a phone book:

- **Linear Search:** Flip page by page → Slow for large books
- **Binary Search:** Open to the middle, decide left or right → Much faster
- Complexity analysis tells you *how much slower* each method gets as the book grows.

## 1.2 Big-O Notation

Big-O describes the **worst-case** upper bound of an algorithm's growth rate.

### Common Complexities (Slowest → Fastest)

| Big-O              | Name         | Example               | 1000 items                                                                                                                                                                             |
|--------------------|--------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| O(1)               | Constant     | Array access by index | 1 op                                                                                                                                                                                   |
| O(log n)           | Logarithmic  | Binary search         | ~10 ops                                                                                                                                                                                |
| O(n)               | Linear       | Simple loop           | 1,000 ops                                                                                                                                                                              |
| O(n log n)         | Linearithmic | Merge sort            | ~10,000 ops                                                                                                                                                                            |
| O(n <sup>2</sup> ) | Quadratic    | Nested loops          | 1,000,000 ops                                                                                                                                                                          |
| O(2 <sup>n</sup> ) | Exponential  | Subsets generation    |  Impossible                                                                                       |
| O(n!)              | Factorial    | Permutations          |   Impossible |

### How to Determine Big-O

```
# O(1) – Constant
def get_first(arr):
    return arr[0] # Always 1 operation
```

```

# O(n) – Linear
def find_max(arr):
    maximum = arr[0]
    for num in arr:          # Loops n times
        if num > maximum:
            maximum = num
    return maximum

# O(n²) – Quadratic
def has_duplicate(arr):
    for i in range(len(arr)):    # n times
        for j in range(i+1, len(arr)): # n times
            if arr[i] == arr[j]:
                return True
    return False

# O(log n) – Logarithmic
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:        # Halves each time
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

```

## Rules for Calculating Big-O

1. **Drop constants:**  $O(2n) \rightarrow O(n)$
2. **Drop lower-order terms:**  $O(n^2 + n) \rightarrow O(n^2)$
3. **Different inputs = different variables:** Two arrays  $\rightarrow O(a \times b)$ , not  $O(n^2)$
4. **Sequential = Add:**  $O(a + b)$
5. **Nested = Multiply:**  $O(a \times b)$

## 1.3 Space Complexity

Space complexity = **Extra memory** your algorithm uses (excluding input).

```

# O(1) Space – In-place
def reverse_in_place(arr):
    left, right = 0, len(arr) - 1

```

```

while left < right:
    arr[left], arr[right] = arr[right], arr[left]
    left += 1
    right -= 1

# O(n) Space – Creates new array
def reverse_copy(arr):
    return arr[::-1] # New list of size n

# O(n) Space – Recursion call stack
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1) # n frames on call stack

```

## 1.4 Amortized Analysis

Some operations are expensive *occasionally* but cheap *on average*.

**Example:** Python `list.append()`

- Usually  $O(1)$
- When the array is full, it resizes (copies everything)  $\rightarrow O(n)$
- But resizing happens so rarely that **amortized** cost =  $O(1)$

## 1.5 Common Mistakes

- ✗ Confusing  $O(n)$  with exact count — Big-O is about *growth rate*, not exact operations
- ✗ Forgetting recursive call stack uses  $O(n)$  space
- ✗ Treating hash map lookups as always  $O(1)$  — Worst case is  $O(n)$  with collisions
- ✗ Ignoring space complexity in interviews
- ✗ Saying “ $O(n^2)$  is always bad” — For  $n \leq 100$ , it’s perfectly fine

## 1.6 Interview Tips

- 💡 Always state **complexity** before and after optimization
- 💡 Say: “The brute force is  $O(n^2)$ , but we can optimize to  $O(n)$  using a hash map”
- 💡 Interviewers love when you compare **tradeoffs** (time vs space)
- 💡 Know that  $O(n \log n)$  is the best for comparison-based sorting

## 1.7 Practice Problems

| # Problem                                                               | Difficulty |
|-------------------------------------------------------------------------|------------|
| 1 What is the Big-O of a loop that runs $n/2$ times?                    | Easy       |
| 2 What is the complexity of 3 nested loops each running $n$ times?      | Easy       |
| 3 Determine Big-O: <code>for i in range(n): for j in range(i, n)</code> | Medium     |
| 4 What's the time complexity of recursive Fibonacci?                    | Medium     |
| 5 Prove that binary search is $O(\log n)$                               | Hard       |

### Answers

1.  $O(n)$  — Drop the constant  $\frac{1}{2}$
2.  $O(n^3)$  — Three nested loops
3.  $O(n^2)$  — Inner loop runs  $n + (n-1) + \dots + 1 = n(n+1)/2 \approx n^2$
4.  $O(2^n)$  — Each call branches into 2, creating a binary tree of depth  $n$
5. Each step halves the array:  $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$ . Steps =  $\log_2(n)$



### Revision Summary

- ✓ Big-O measures worst-case growth rate
- ✓ Drop constants and lower-order terms
- ✓ Nested loops  $\rightarrow$  Multiply; Sequential  $\rightarrow$  Add
- ✓ Space = extra memory used (watch for recursion stack!)
- ✓ Amortized = average cost over many operations
- ✓ Always analyze BOTH time and space in interviews

*Next: Module 2 — Arrays & Strings  $\rightarrow$*

# Module 2: Arrays & Strings

## 2.1 What is an Array?

**Definition:** An array is a **contiguous block of memory** storing elements of the same type, accessible by index.

### Real-Life Analogy

Think of a **row of lockers** in a school. Each locker:

- Has a **number** (index)
- Holds **one item** (value)
- You can instantly open locker #5 without checking 1-4

## Python Implementation

Python uses **lists** (dynamic arrays) — they auto-resize.

```
# Creating arrays
arr = [1, 2, 3, 4, 5]           # List literal
arr = [0] * 10                  # Array of 10 zeros
arr = list(range(1, 11))        # [1, 2, ..., 10]

# Core operations & their complexity
arr[0]                          # Access: O(1)
arr.append(6)                   # Append: O(1) amortized
arr.insert(2, 99)               # Insert at index: O(n) – shifts elements
arr.pop()                       # Remove last: O(1)
arr.pop(2)                      # Remove at index: O(n) – shifts elements
arr.remove(99)                  # Remove by value: O(n) – search + shift
len(arr)                        # Length: O(1)
99 in arr                       # Search: O(n)
arr.sort()                      # Sort: O(n log n)
arr.reverse()                   # Reverse: O(n)
```

## Operations Complexity Table

| Operation       | Time | Space |
|-----------------|------|-------|
| Access by index | O(1) | O(1)  |



| Operation                  | Time             | Space  |
|----------------------------|------------------|--------|
| Search (unsorted)          | $O(n)$           | $O(1)$ |
| Search (sorted, binary)    | $O(\log n)$      | $O(1)$ |
| Insert at end              | $O(1)$ amortized | $O(1)$ |
| Insert at beginning/middle | $O(n)$           | $O(1)$ |
| Delete at end              | $O(1)$           | $O(1)$ |
| Delete at beginning/middle | $O(n)$           | $O(1)$ |

## 2.2 Essential Array Techniques

### Technique 1: Two Pointers

Use when: sorted array, pair finding, partitioning.

```
def two_sum_sorted(arr, target):
    """Find two numbers that add up to target in a SORTED array."""
    left, right = 0, len(arr) - 1
    while left < right:
        current_sum = arr[left] + arr[right]
        if current_sum == target:
            return [left, right]
        elif current_sum < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]

# Time:  $O(n)$  | Space:  $O(1)$ 
```

### Technique 2: Sliding Window

Use when: contiguous subarray/substring problems.

```
def max_sum_subarray(arr, k):
    """Find maximum sum of any subarray of size k."""
    if len(arr) < k:
        return -1

    window_sum = sum(arr[:k])
    max_sum = window_sum
```

```
for i in range(k, len(arr)):
    window_sum += arr[i] - arr[i - k] # Slide the window
    max_sum = max(max_sum, window_sum)

return max_sum
```

# Time:  $O(n)$  | Space:  $O(1)$

## Technique 3: Prefix Sum

Use when: range sum queries.

```
def build_prefix_sum(arr):
    """Build prefix sum array for  $O(1)$  range queries."""
    prefix = [0] * (len(arr) + 1)
    for i in range(len(arr)):
        prefix[i + 1] = prefix[i] + arr[i]
    return prefix
```

```
def range_sum(prefix, left, right):
    """Sum of arr[left..right] in  $O(1)$ ."""
    return prefix[right + 1] - prefix[left]
```

# Build:  $O(n)$  | Query:  $O(1)$  | Space:  $O(n)$

## Technique 4: Kadane's Algorithm

Use when: maximum subarray sum.

```
def max_subarray_sum(arr):
    """Find the maximum sum of any contiguous subarray."""
    max_sum = current_sum = arr[0]
    for num in arr[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum
```

# Time:  $O(n)$  | Space:  $O(1)$

---

## 2.3 Strings in Python

Strings are **immutable** arrays of characters.

```
s = "hello"
s[0]          # 'h' - Access: O(1)
s[1:4]        # 'ell' - Slicing: O(k)
s + " world"  # Concatenation: O(n+m) - creates NEW string
len(s)        # O(1)
s.find("ll")  # O(n)
s.replace("l", "L") # O(n)
s[::-1]       # Reverse: O(n)

# ⚠ String concatenation in a loop is O(n²)!
# BAD:
result = ""
for char in "hello":
    result += char          # Creates new string each time → O(n²)

# GOOD:
result = []
for char in "hello":
    result.append(char)     # O(1) amortized
result = "".join(result)   # O(n) total
```

## Essential String Techniques

```
# Character frequency counting
from collections import Counter
freq = Counter("abracadabra") # {'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1}

# Check anagram
def is_anagram(s1, s2):
    return Counter(s1) == Counter(s2)
# Time: O(n) | Space: O(1) - at most 26 lowercase letters

# Check palindrome
def is_palindrome(s):
    return s == s[::-1]

# Or with two pointers:
def is_palindrome_tp(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
```

```
        return False
    left += 1
    right -= 1
    return True
```

## 2.4 Pattern Recognition Tips

- 🔍 “Find pair/triplet” → Two Pointers or HashMap
- 🔍 “Contiguous subarray” → Sliding Window or Prefix Sum
- 🔍 “Maximum/minimum subarray” → Kadane’s Algorithm
- 🔍 “Subarray of fixed size k” → Sliding Window (fixed)
- 🔍 “In-place operation” → Two Pointers
- 🔍 “Anagram/permutation” → Character frequency (Counter)
- 🔍 “Palindrome” → Two Pointers from both ends

## 2.5 Common Mistakes

- ❌ Off-by-one errors in loop bounds (use `range(len(arr))` carefully)
- ❌ Modifying a list while iterating — iterate over a copy instead
- ❌ String concatenation in loops — use `"".join()` instead
- ❌ Forgetting arrays are 0-indexed in Python
- ❌ Not handling empty arrays — always check `if not arr: return`
- ❌ Using `arr2 = arr1` — this copies the *reference*, not the array. Use `arr2 = arr1[:]` or `arr2 = list(arr1)`

## 2.6 Interview Tips

- 💡 **Clarify:** Is the array sorted? Are there duplicates? Can values be negative?
- 💡 **Edge cases:** Empty array, single element, all same elements
- 💡 **In-place vs extra space:** Ask if you can modify the input
- 💡 **Mention the brute force first**, then optimize
- 💡 **Two Sum** is the #1 most-asked array problem — master it cold

## 2.7 Practice Problems

| # | Problem               | Technique | Difficulty |
|---|-----------------------|-----------|------------|
| 1 | Two Sum (LeetCode #1) | HashMap   | Easy       |

| # Problem                                                | Technique            | Difficulty |
|----------------------------------------------------------|----------------------|------------|
| 2 Best Time to Buy and Sell Stock (LC #121)              | Kadane's variant     | Easy       |
| 3 Container With Most Water (LC #11)                     | Two Pointers         | Medium     |
| 4 Longest Substring Without Repeating Characters (LC #3) | Sliding Window       | Medium     |
| 5 Trapping Rain Water (LC #42)                           | Two Pointers / Stack | Hard       |

## Solution: Two Sum (LC #1)

```
def two_sum(nums, target):
    """Return indices of two numbers that add up to target."""
    seen = {} # value → index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

# Time:  $O(n)$  | Space:  $O(n)$

## Solution: Best Time to Buy and Sell Stock (LC #121)

```
def max_profit(prices):
    """Find max profit from one buy and one sell."""
    min_price = float('inf')
    max_profit = 0
    for price in prices:
        min_price = min(min_price, price)
        max_profit = max(max_profit, price - min_price)
    return max_profit
```

# Time:  $O(n)$  | Space:  $O(1)$



## Revision Summary

- ✓ Arrays = contiguous memory,  $O(1)$  access by index
- ✓ Python lists are dynamic arrays (auto-resize)
- ✓ Two Pointers: sorted arrays, pair finding
- ✓ Sliding Window: contiguous subarray problems

- ✓ Prefix Sum: range sum queries in  $O(1)$
  - ✓ Kadane's: maximum subarray sum in  $O(n)$
  - ✓ Strings are immutable – use lists + `join()` for building
  - ✓ Counter for frequency problems
  - ✓ Always handle edge cases: empty, single element, duplicates
- 

*Next: Module 3 — Linked Lists →*

---

# Module 3: Linked Lists

## 3.1 What is a Linked List?

**Definition:** A linked list is a linear data structure where elements (**nodes**) are stored in **non-contiguous memory**. Each node contains data and a **pointer** to the next node.

### Real-Life Analogy

Think of a **treasure hunt**. Each clue (node) has:

- A **message** (data)
- **Directions to the next clue** (pointer)

You must follow clues in order — you can’t jump to clue #5 without going through 1-4.

### Array vs Linked List

| Feature             | Array          | Linked List            |
|---------------------|----------------|------------------------|
| Memory              | Contiguous     | Scattered              |
| Access by index     | O(1)           | O(n)                   |
| Insert at beginning | O(n)           | O(1)                   |
| Insert at end       | O(1) amortized | O(n) or O(1) with tail |
| Delete              | O(n)           | O(1) if node known     |
| Memory overhead     | None           | Extra pointer per node |

## 3.2 Singly Linked List — Implementation

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.size = 0

    def prepend(self, val):
```

```
"""Insert at the beginning – O(1)."""
new_node = ListNode(val, self.head)
self.head = new_node
self.size += 1
```

```
def append(self, val):
    """Insert at the end – O(n)."""
    new_node = ListNode(val)
    if not self.head:
        self.head = new_node
    else:
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node
    self.size += 1
```

```
def delete(self, val):
    """Delete first occurrence of val – O(n)."""
    if not self.head:
        return
    if self.head.val == val:
        self.head = self.head.next
        self.size -= 1
        return
    curr = self.head
    while curr.next:
        if curr.next.val == val:
            curr.next = curr.next.next
            self.size -= 1
            return
        curr = curr.next
```

```
def search(self, val):
    """Search for val – O(n)."""
    curr = self.head
    while curr:
        if curr.val == val:
            return True
        curr = curr.next
    return False
```

```
def display(self):
    """Print the list – O(n)."""
    elements = []
    curr = self.head
    while curr:
        elements.append(str(curr.val))
```



```
        curr = curr.next
    print(" → ".join(elements) + " → None")
```

# Usage

```
ll = SinglyLinkedList()
ll.append(1)
ll.append(2)
ll.append(3)
ll.prepend(0)
ll.display()  # 0 → 1 → 2 → 3 → None
```

## 3.3 Doubly Linked List

```
class DListNode:
    def __init__(self, val=0, prev=None, next=None):
        self.val = val
        self.prev = prev
        self.next = next

class DoublyLinkedList:
    def __init__(self):
        # Sentinel nodes simplify edge cases
        self.head = DListNode(0)  # dummy head
        self.tail = DListNode(0)  # dummy tail
        self.head.next = self.tail
        self.tail.prev = self.head

    def add_to_front(self, val):
        """Insert after dummy head – O(1)."""
        node = DListNode(val, self.head, self.head.next)
        self.head.next.prev = node
        self.head.next = node

    def add_to_end(self, val):
        """Insert before dummy tail – O(1)."""
        node = DListNode(val, self.tail.prev, self.tail)
        self.tail.prev.next = node
        self.tail.prev = node

    def remove(self, node):
        """Remove a specific node – O(1)."""
        node.prev.next = node.next
        node.next.prev = node.prev
```

```
def display(self):
    elements = []
    curr = self.head.next
    while curr != self.tail:
        elements.append(str(curr.val))
        curr = curr.next
    print(" ⇌ ".join(elements))
```

💡 **Pro Tip:** Sentinel (dummy) nodes eliminate all edge cases for empty lists and boundary operations. LRU Cache uses a doubly linked list with sentinels.

## 3.4 Essential Linked List Techniques

### Technique 1: Fast and Slow Pointers (Floyd's)

```
def has_cycle(head):
    """Detect if linked list has a cycle – O(n) time, O(1) space."""
    slow = fast = head
    while fast and fast.next:
        slow = slow.next          # Move 1 step
        fast = fast.next.next     # Move 2 steps
        if slow == fast:
            return True
    return False

def find_middle(head):
    """Find the middle node – O(n) time, O(1) space."""
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow # slow is at the middle
```

### Technique 2: Reverse a Linked List

```
def reverse_list(head):
    """Reverse in-place – O(n) time, O(1) space."""
    prev = None
    curr = head
    while curr:
        next_temp = curr.next # Save next
        curr.next = prev      # Reverse pointer
```

```

    prev = curr          # Move prev forward
    curr = next_temp     # Move curr forward
return prev # New head

```

# Recursive version –  $O(n)$  time,  $O(n)$  space (call stack)

```

def reverse_recursive(head):
    if not head or not head.next:
        return head
    new_head = reverse_recursive(head.next)
    head.next.next = head
    head.next = None
    return new_head

```

## Technique 3: Dummy Head Node

```

def merge_two_sorted(l1, l2):
    """Merge two sorted lists –  $O(n+m)$  time,  $O(1)$  space."""
    dummy = ListNode(0) # Dummy head simplifies code
    curr = dummy

    while l1 and l2:
        if l1.val <= l2.val:
            curr.next = l1
            l1 = l1.next
        else:
            curr.next = l2
            l2 = l2.next
        curr = curr.next

    curr.next = l1 or l2 # Attach remaining
    return dummy.next   # Skip dummy

```

## 3.5 Pattern Recognition Tips

-  “Cycle detection” → Fast/Slow pointers
-  “Find middle” → Fast/Slow pointers
-  “Reverse” → Three-pointer technique (prev, curr, next)
-  “Merge sorted lists” → Dummy head + comparison
-  “Remove nth from end” → Two pointers with gap of n
-  “Palindrome check” → Find middle → Reverse second half → Compare

## 3.6 Common Mistakes

- ✗ **Losing the head pointer** — always save it or use a dummy
- ✗ **Not handling None** — check `if curr:` before accessing `curr.val`
- ✗ **Infinite loops** — forgetting to advance the pointer
- ✗ **Memory leaks** — in languages with manual memory (Python handles this)
- ✗ **Off-by-one** when finding `nth` from end

## 3.7 Interview Tips

- 💡 **Draw it out** — Always sketch the linked list on paper/whiteboard
- 💡 **Use a dummy head** for merge/insert problems — eliminates edge cases
- 💡 **Clarify:** Singly or doubly? Sorted? Has cycle?
- 💡 **Recursive vs iterative:** Know both, but iterative is usually preferred ( $O(1)$  space)
- 💡 **Reversing a linked list** is the most fundamental LL operation — practice until automatic

## 3.8 Practice Problems

| # | Problem                           | Technique             | Difficulty |
|---|-----------------------------------|-----------------------|------------|
| 1 | Reverse Linked List (LC #206)     | Three pointers        | Easy       |
| 2 | Linked List Cycle (LC #141)       | Fast/Slow             | Easy       |
| 3 | Merge Two Sorted Lists (LC #21)   | Dummy head            | Easy       |
| 4 | Remove Nth Node From End (LC #19) | Two pointers with gap | Medium     |
| 5 | LRU Cache (LC #146)               | Doubly LL + HashMap   | Hard       |

### Solution: Remove Nth Node From End (LC #19)

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy

    # Advance fast by n+1 steps
    for _ in range(n + 1):
        fast = fast.next

    # Move both until fast reaches end
    while fast:
        fast = fast.next
        slow = slow.next
```

```
slow.next = slow.next.next # Skip the nth node
return dummy.next
```

# Time:  $O(n)$  | Space:  $O(1)$



## Revision Summary

- ✓ Linked list = nodes with data + pointer, non-contiguous memory
- ✓ Singly LL: each node points to next only
- ✓ Doubly LL: each node points to prev AND next
- ✓ Sentinel/dummy nodes eliminate boundary edge cases
- ✓ Fast/Slow pointers: cycle detection, finding middle
- ✓ Reverse: prev/curr/next three-pointer swap
- ✓ Always draw diagrams when solving LL problems
- ✓  $O(1)$  insert/delete at known position vs  $O(n)$  for arrays

*Next: Module 4 — Stacks & Queues →*

# Module 4: Stacks & Queues

---

## 4.1 Stack — Last In, First Out (LIFO)

**Definition:** A stack is a linear data structure where elements are added and removed from the **same end** (top).

### Real-Life Analogy

A **stack of plates**: You can only add or remove the plate on **top**. The last plate placed is the first one removed.

### Python Implementation

```
# Method 1: Using Python list (most common)
stack = []
stack.append(1)      # Push: O(1)
stack.append(2)
stack.append(3)
top = stack[-1]      # Peek: O(1) → 3
item = stack.pop()   # Pop: O(1) → 3
is_empty = len(stack) == 0 # O(1)

# Method 2: Using collections.deque (slightly faster)
from collections import deque
stack = deque()
stack.append(1)       # Push
stack.pop()           # Pop

# Method 3: Custom Stack Class
class Stack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self._items.pop()
```

```
def peek(self):
    if self.is_empty():
        raise IndexError("Stack is empty")
    return self._items[-1]

def is_empty(self):
    return len(self._items) == 0

def size(self):
    return len(self._items)
```

## Stack Operations Complexity

| Operation | Time | Space |
|-----------|------|-------|
| Push      | O(1) | O(1)  |
| Pop       | O(1) | O(1)  |
| Peek      | O(1) | O(1)  |
| Search    | O(n) | O(1)  |
| isEmpty   | O(1) | O(1)  |

## 4.2 Classic Stack Problems

### Valid Parentheses (LC #20)

```
def is_valid(s):
    """Check if parentheses are balanced."""
    stack = []
    mapping = {'(': ')', '{': '}', '[': ']'}

    for char in s:
        if char in mapping:
            if not stack or stack[-1] != mapping[char]:
                return False
            stack.pop()
        else:
            stack.append(char)

    return len(stack) == 0

# Time: O(n) | Space: O(n)
```

## Min Stack (LC #155)

```
class MinStack:
    """Stack that supports O(1) getMin."""
    def __init__(self):
        self.stack = []
        self.min_stack = [] # Tracks minimums

    def push(self, val):
        self.stack.append(val)
        min_val = min(val, self.min_stack[-1] if self.min_stack else val)
        self.min_stack.append(min_val)

    def pop(self):
        self.stack.pop()
        self.min_stack.pop()

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1]

# All operations: O(1) time, O(n) space
```

## Monotonic Stack — Next Greater Element

```
def next_greater_element(nums):
    """For each element, find the next element that is greater."""
    result = [-1] * len(nums)
    stack = [] # Stack of indices

    for i in range(len(nums)):
        while stack and nums[i] > nums[stack[-1]]:
            idx = stack.pop()
            result[idx] = nums[i]
        stack.append(i)

    return result

# nums = [2, 1, 2, 4, 3]
# result = [4, 2, 4, -1, -1]
# Time: O(n) | Space: O(n)
```



## 4.3 Queue — First In, First Out (FIFO)

**Definition:** A queue is a linear data structure where elements are added at the **rear** and removed from the **front**.

### Real-Life Analogy

**A line at a coffee shop:** First person in line gets served first. New customers join at the back.

### Python Implementation

```
# Method 1: Using deque (BEST – O(1) both ends)
from collections import deque
queue = deque()
queue.append(1)      # Enqueue (add to rear): O(1)
queue.append(2)
queue.append(3)
front = queue[0]     # Peek: O(1) → 1
item = queue.popleft() # Dequeue (remove from front): O(1)

# ⚠️ DO NOT use list for queues!
# list.pop(0) is O(n) because it shifts all elements

# Method 2: Custom Queue
class Queue:
    def __init__(self):
        self._items = deque()

    def enqueue(self, item):
        self._items.append(item)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self._items.popleft()

    def peek(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self._items[0]

    def is_empty(self):
        return len(self._items) == 0

    def size(self):
```

```
return len(self._items)
```

## Queue Operations Complexity (using deque)

| Operation | Time | Space |
|-----------|------|-------|
| Enqueue   | O(1) | O(1)  |
| Dequeue   | O(1) | O(1)  |
| Peek      | O(1) | O(1)  |
| Search    | O(n) | O(1)  |
| isEmpty   | O(1) | O(1)  |

## 4.4 Special Queue Types

### Circular Queue

```
class CircularQueue:
    """Fixed-size queue using an array with wrap-around."""
    def __init__(self, capacity):
        self.queue = [None] * capacity
        self.capacity = capacity
        self.front = 0
        self.rear = -1
        self.size = 0

    def enqueue(self, item):
        if self.size == self.capacity:
            raise OverflowError("Queue is full")
        self.rear = (self.rear + 1) % self.capacity
        self.queue[self.rear] = item
        self.size += 1

    def dequeue(self):
        if self.size == 0:
            raise IndexError("Queue is empty")
        item = self.queue[self.front]
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
        return item
```

## Queue using Two Stacks (Interview Classic!)

```
class QueueUsingStacks:
    """Implement queue using two stacks – amortized O(1)."""
    def __init__(self):
        self.in_stack = [] # For push
        self.out_stack = [] # For pop

    def enqueue(self, val):
        self.in_stack.append(val)

    def dequeue(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        if not self.out_stack:
            raise IndexError("Queue is empty")
        return self.out_stack.pop()

    def peek(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        return self.out_stack[-1]

# Amortized O(1) for all operations
```

---

## 4.5 Pattern Recognition Tips

- 🔍 “Matching/balancing” (parentheses, tags) → Stack
- 🔍 “Next greater/smaller element” → Monotonic Stack
- 🔍 “Undo/redo” → Two Stacks
- 🔍 “Process in order (BFS)” → Queue
- 🔍 “Evaluate expression” → Stack
- 🔍 “Sliding window maximum” → Monotonic Deque
- 🔍 “Design data structure” → Combine Stack/Queue with HashMap

---

## 4.6 Common Mistakes

- ❌ Using `list.pop(0)` for queue — use `deque.popleft()` instead ( $O(n)$  vs  $O(1)$ )
- ❌ Forgetting to check empty before pop/peek

- ✗ **Confusing LIFO/FIFO** — Stack = LIFO, Queue = FIFO
  - ✗ **Not considering monotonic stack** for “next greater” problems
  - ✗ **Overcomplicating** — most stack problems need just `append()` and `pop()`
- 

## 4.7 Interview Tips

- 💡 **Stacks** are the most versatile interview data structure — they solve 15+ problem types
  - 💡 **Know monotonic stack** — it appears in Amazon, Google, and Meta interviews frequently
  - 💡 **“Can you do it in one pass?”** → Usually means stack
  - 💡 **Implement queue with stacks** and vice versa — classic design question
  - 💡 **BFS** always uses a **queue**, **DFS** uses a **stack** (or recursion = implicit stack)
- 

## 4.8 Practice Problems

| # | Problem                                 | Type            | Difficulty |
|---|-----------------------------------------|-----------------|------------|
| 1 | Valid Parentheses (LC #20)              | Stack           | Easy       |
| 2 | Implement Queue using Stacks (LC #232)  | Design          | Easy       |
| 3 | Min Stack (LC #155)                     | Stack Design    | Medium     |
| 4 | Daily Temperatures (LC #739)            | Monotonic Stack | Medium     |
| 5 | Largest Rectangle in Histogram (LC #84) | Monotonic Stack | Hard       |

### Solution: Daily Temperatures (LC #739)

```
def daily_temperatures(temperatures):  
    """How many days until a warmer temperature."""  
    result = [0] * len(temperatures)  
    stack = [] # Stack of indices  
  
    for i, temp in enumerate(temperatures):  
        while stack and temp > temperatures[stack[-1]]:  
            prev_idx = stack.pop()  
            result[prev_idx] = i - prev_idx  
        stack.append(i)  
  
    return result  
  
# Time: O(n) | Space: O(n)
```

---



## Revision Summary

- ✓ Stack = LIFO (Last In, First Out) – use list or deque
- ✓ Queue = FIFO (First In, First Out) – ALWAYS use deque
- ✓ Monotonic stack: maintains increasing/decreasing order
- ✓ Valid parentheses = classic stack problem
- ✓ Queue with two stacks = amortized  $O(1)$
- ✓ Stack for DFS, Queue for BFS
- ✓ Min Stack: maintain parallel stack tracking minimums
- ✓ Never use `list.pop(0)` – it's  $O(n)$ !

---

*Next: Module 5 — Hashing & HashMaps →*

---

# Module 5: Hashing & HashMaps

---

## 5.1 What is Hashing?

**Definition:** Hashing converts a key into a fixed-size integer (hash) using a **hash function**, which maps to an index in an array (hash table) for  $O(1)$  average-case lookups.



### Real-Life Analogy

Think of a **library catalog system**: instead of searching every shelf, you look up the book's code (hash), and it tells you the exact shelf and position. Instant access.

### How it Works

Key → Hash Function → Index → Value

"apple" → hash("apple") → 42 → bucket[42] = "A fruit"

### Python Implementation — Hash Table from Scratch

```
class HashTable:
    def __init__(self, size=1024):
        self.size = size
        self.table = [[] for _ in range(size)] # Chaining

    def _hash(self, key):
        return hash(key) % self.size

    def put(self, key, value):
        idx = self._hash(key)
        for i, (k, v) in enumerate(self.table[idx]):
            if k == key:
                self.table[idx][i] = (key, value) # Update
                return
        self.table[idx].append((key, value)) # Insert

    def get(self, key):
        idx = self._hash(key)
        for k, v in self.table[idx]:
            if k == key:
                return v
        raise KeyError(key)
```

```
def remove(self, key):
    idx = self._hash(key)
    for i, (k, v) in enumerate(self.table[idx]):
        if k == key:
            self.table[idx].pop(i)
            return
    raise KeyError(key)
```

## Collision Handling

| Method                   | How it Works                 | Pros               | Cons                      |
|--------------------------|------------------------------|--------------------|---------------------------|
| <b>Chaining</b>          | Each bucket is a linked list | Simple, never full | Extra memory for pointers |
| <b>Open Addressing</b>   | Find next empty slot         | Better cache perf  | Complex deletion          |
| <b>Linear Probing</b>    | Check next slot              | Simple             | Clustering                |
| <b>Quadratic Probing</b> | Check $i^2$ slots away       | Less clustering    | May miss slots            |

## 5.2 Python Dict & Set — Built-in Hash Tables

```
# Dictionary (HashMap) – key:value pairs
d = {}
d["name"] = "Alice"      # Insert: O(1) avg
print(d["name"])         # Access: O(1) avg
del d["name"]            # Delete: O(1) avg
"name" in d               # Search: O(1) avg

# Useful dict methods
d.get("key", "default")  # Returns default if key missing
d.setdefault("key", []).append(1) # Initialize + append
d.items()                # Key-value pairs
d.keys()                 # All keys
d.values()               # All values

# defaultdict – auto-initializes missing keys
from collections import defaultdict
graph = defaultdict(list)
graph["A"].append("B")   # No KeyError!

counter = defaultdict(int)
counter["a"] += 1        # Auto-initializes to 0

# Counter – frequency counting
from collections import Counter
```

```
freq = Counter("abracadabra") # {'a': 5, 'b': 2, ...}
freq.most_common(2)          # [('a', 5), ('b', 2)]

# Set – unique elements, O(1) lookup
s = {1, 2, 3}
s.add(4)                      # O(1)
s.remove(4)                   # O(1), raises KeyError if missing
s.discard(4)                  # O(1), no error if missing
4 in s                        # O(1)
s1 & s2                        # Intersection
s1 | s2                       # Union
s1 - s2                       # Difference
```

## Operations Complexity

| Operation | Average | Worst Case |
|-----------|---------|------------|
| Insert    | O(1)    | O(n)       |
| Lookup    | O(1)    | O(n)       |
| Delete    | O(1)    | O(n)       |
| Iteration | O(n)    | O(n)       |

Worst case O(n) happens with many hash collisions — extremely rare with good hash functions.

## 5.3 Essential Hashing Patterns

### Pattern 1: Frequency Counting

```
def top_k_frequent(nums, k):
    """Find k most frequent elements – O(n)."""
    count = Counter(nums)
    # Bucket sort approach
    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in count.items():
        buckets[freq].append(num)

    result = []
    for i in range(len(buckets) - 1, -1, -1):
        result.extend(buckets[i])
        if len(result) >= k:
            return result[:k]
    return result
```



```
# Time: O(n) | Space: O(n)
```

## Pattern 2: Two Sum (HashMap Approach)

```
def two_sum(nums, target):  
    """Find indices of two numbers summing to target."""  
    seen = {} # value → index  
    for i, num in enumerate(nums):  
        complement = target - num  
        if complement in seen:  
            return [seen[complement], i]  
        seen[num] = i
```

```
# Time: O(n) | Space: O(n)
```

## Pattern 3: Group Anagrams

```
def group_anagrams(strs):  
    """Group strings that are anagrams of each other."""  
    groups = defaultdict(list)  
    for s in strs:  
        key = tuple(sorted(s)) # Sorted chars as key  
        groups[key].append(s)  
    return list(groups.values())
```

```
# Time: O(n * k log k) where k = max string length
```

```
# Space: O(n * k)
```

## Pattern 4: Subarray Sum Equals K (Prefix Sum + HashMap)

```
def subarray_sum(nums, k):  
    """Count subarrays with sum equal to k."""  
    count = 0  
    prefix_sum = 0  
    prefix_map = {0: 1} # Base case  
  
    for num in nums:  
        prefix_sum += num  
        if prefix_sum - k in prefix_map:  
            count += prefix_map[prefix_sum - k]
```

```
        prefix_map[prefix_sum] = prefix_map.get(prefix_sum, 0) + 1

    return count

# Time: O(n) | Space: O(n)
```

## 5.4 Pattern Recognition Tips

- 🔍 “Find duplicates” → HashSet
- 🔍 “Count frequency” → Counter / defaultdict(int)
- 🔍 “Group items” → defaultdict(list) with a clever key
- 🔍 “Two Sum / pair finding” → HashMap (complement lookup)
- 🔍 “Subarray sum” → Prefix Sum + HashMap
- 🔍 “First non-repeating” → OrderedDict or Counter
- 🔍 “Check if exists in O(1)” → HashSet

## 5.5 Common Mistakes

- ❌ **Mutable keys** — lists can’t be dict keys (use tuples instead)
- ❌ **KeyError** — use `dict.get(key, default)` or `defaultdict`
- ❌ **Assuming O(1) is guaranteed** — worst case is O(n) with collisions
- ❌ **Ignoring space** — HashMaps use O(n) extra space
- ❌ **Not considering Counter** — reinventing frequency counting manually

## 5.6 Interview Tips

- 💡 **HashMap is the #1 tool** for optimizing brute force from O(n<sup>2</sup>) to O(n)
- 💡 Always ask: “Can I trade space for time using a hash map?”
- 💡 **defaultdict** saves lines of code in interviews — use it
- 💡 Know the difference between `dict`, `set`, `Counter`, `defaultdict`
- 💡 **Subarray sum problems** almost always use prefix sum + hashmap

## 5.7 Practice Problems

| # | Problem                      | Pattern | Difficulty |
|---|------------------------------|---------|------------|
| 1 | Two Sum (LC #1)              | HashMap | Easy       |
| 2 | Contains Duplicate (LC #217) | HashSet | Easy       |

| # | Problem                         | Pattern             | Difficulty |
|---|---------------------------------|---------------------|------------|
| 3 | Group Anagrams (LC #49)         | HashMap grouping    | Medium     |
| 4 | Subarray Sum Equals K (LC #560) | Prefix + HashMap    | Medium     |
| 5 | LRU Cache (LC #146)             | HashMap + Doubly LL | Hard       |



## Revision Summary

- ✓ Hashing = key → hash function → index →  $O(1)$  access
- ✓ Collisions handled by chaining or open addressing
- ✓ Python dict and set are hash tables under the hood
- ✓ Counter for frequency, defaultdict for auto-init
- ✓ HashMap optimizes  $O(n^2) \rightarrow O(n)$  in many problems
- ✓ Prefix Sum + HashMap = subarray sum pattern
- ✓ Keys must be immutable (str, int, tuple – NOT list)
- ✓ Average  $O(1)$ , worst case  $O(n)$  – but rare in practice

*Next: Module 6 — Recursion & Backtracking →*

# Module 6: Recursion & Backtracking

## 6.1 What is Recursion?

**Definition:** Recursion is when a function **calls itself** to solve a smaller version of the same problem, until it hits a **base case**.



### Real-Life Analogy

**Russian nesting dolls (Matryoshka):** Open a doll → find a smaller doll inside → keep opening until you reach the smallest doll (base case) → now close them back up (return values).

## Anatomy of a Recursive Function

```
def recursive_function(input):  
    # 1. BASE CASE – when to stop  
    if input is smallest:  
        return result  
  
    # 2. RECURSIVE CASE – break into smaller subproblem  
    return recursive_function(smaller_input)
```

## Classic Examples

```
# Factorial:  $n! = n \times (n-1)!$   
def factorial(n):  
    if n <= 1:          # Base case  
        return 1  
    return n * factorial(n - 1) # Recursive case  
# Time:  $O(n)$  | Space:  $O(n)$  – call stack  
  
# Fibonacci:  $F(n) = F(n-1) + F(n-2)$   
def fibonacci(n):  
    if n <= 1:          # Base case  
        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)  
# Time:  $O(2^n)$  🔥 | Space:  $O(n)$   
# This is TERRIBLE – use DP instead (Module 11)  
  
# Power:  $x^n$ 
```

```
def power(x, n):
    if n == 0:
        return 1
    if n < 0:
        return 1 / power(x, -n)
    if n % 2 == 0:
        half = power(x, n // 2)
        return half * half
    return x * power(x, n - 1)
# Time: O(log n) | Space: O(log n)
```

## Recursion vs Iteration

| Feature     | Recursion                       | Iteration      |
|-------------|---------------------------------|----------------|
| Readability | Often cleaner                   | Can be verbose |
| Space       | O(n) call stack                 | O(1) usually   |
| Speed       | Overhead per call               | Faster         |
| Use when    | Trees, graphs, divide & conquer | Simple loops   |
| Risk        | Stack overflow                  | None           |

## 6.2 Backtracking

**Definition:** Backtracking = Recursion + **undo choices**. Explore all possibilities, and when a path fails, **backtrack** (undo the last choice) and try the next option.

### Real-Life Analogy

A **maze**: Walk forward, hit a dead end → walk back to the last fork → try a different path → repeat until you find the exit.

### Backtracking Template

```
def backtrack(candidates, path, result):
    # Base case: found a valid solution
    if is_valid_solution(path):
        result.append(path[:]) # Add copy of current path
        return

    for candidate in candidates:
        if is_valid_candidate(candidate):
            path.append(candidate) # CHOOSE
            backtrack(next_candidates, path, result) # EXPLORE
```

```
path.pop()
```

```
# UN-CHOOSE (backtrack)
```

## 6.3 Classic Backtracking Problems

### Subsets (LC #78)

```
def subsets(nums):
    """Generate all subsets of nums."""
    result = []

    def backtrack(start, path):
        result.append(path[:]) # Add current subset

        for i in range(start, len(nums)):
            path.append(nums[i])    # Choose
            backtrack(i + 1, path)  # Explore
            path.pop()              # Un-choose

    backtrack(0, [])
    return result

# nums = [1,2,3] → [[], [1], [1,2], [1,2,3], [1,3], [2], [2,3], [3]]
# Time: O(n × 2^n) | Space: O(n)
```

### Permutations (LC #46)

```
def permutations(nums):
    """Generate all permutations."""
    result = []

    def backtrack(path, remaining):
        if not remaining:
            result.append(path[:])
            return

        for i in range(len(remaining)):
            path.append(remaining[i])
            backtrack(path, remaining[:i] + remaining[i+1:])
            path.pop()

    backtrack([], nums)
    return result
```

```
# nums = [1,2,3] → [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
# Time: O(n × n!) | Space: O(n)
```

## Combination Sum (LC #39)

```
def combination_sum(candidates, target):
    """Find all combinations that sum to target (reuse allowed)."""
    result = []

    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(path[:])
            return
        if remaining < 0:
            return

        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i]) # i, not i+1 (reuse)
            path.pop()

    candidates.sort()
    backtrack(0, [], target)
    return result

# Time: O(n^(T/M)) where T=target, M=min candidate
```

## N-Queens (LC #51)

```
def solve_n_queens(n):
    """Place n queens on nxn board so none attack each other."""
    result = []
    cols = set()
    pos_diag = set() # row + col
    neg_diag = set() # row - col

    board = [["."] * n for _ in range(n)]

    def backtrack(row):
        if row == n:
            result.append(["".join(r) for r in board])
            return

        for col in range(n):
            if col in cols or row + col in pos_diag or row - col in neg_diag:
                continue
            board[row][col] = "Q"
            cols.add(col)
            pos_diag.add(row + col)
            neg_diag.add(row - col)
            backtrack(row + 1)
            board[row][col] = "."
            cols.remove(col)
            pos_diag.remove(row + col)
            neg_diag.remove(row - col)
```

```

for col in range(n):
    if col in cols or (row + col) in pos_diag or (row - col) in neg_diag:
        continue

    # Place queen
    cols.add(col)
    pos_diag.add(row + col)
    neg_diag.add(row - col)
    board[row][col] = "Q"

    backtrack(row + 1)

    # Remove queen (backtrack)
    cols.remove(col)
    pos_diag.remove(row + col)
    neg_diag.remove(row - col)
    board[row][col] = "."

backtrack(0)
return result

# Time: O(n!) | Space: O(n²)

```

## 6.4 Pattern Recognition Tips

- 🔍 “Generate all subsets” → Backtracking (include/exclude each element)
- 🔍 “Generate all permutations” → Backtracking (swap or remaining list)
- 🔍 “Find all combinations summing to X” → Backtracking with sum tracking
- 🔍 “Place items with constraints” → Backtracking (N-Queens, Sudoku)
- 🔍 “Word search in grid” → Backtracking with visited set
- 🔍 “Palindrome partitioning” → Backtracking with palindrome check

## Decision Tree for Recursion vs Backtracking vs DP

```

Problem requires exploring all possibilities?
├── YES → Are there overlapping subproblems?
│   ├── YES → Dynamic Programming (Module 11)
│   └── NO → Do you need to undo choices?
│       ├── YES → Backtracking
│       └── NO → Simple Recursion
└── NO → Greedy or other approach

```



## 6.5 Common Mistakes

- ✗ Forgetting the base case → infinite recursion → stack overflow
  - ✗ Not making a copy of path: `result.append(path[:])` not `result.append(path)`
  - ✗ Forgetting to undo the choice after recursive call (the “backtrack” step)
  - ✗ Not sorting candidates when you need to skip duplicates
  - ✗ Modifying the loop variable inside the loop
  - ✗ Not setting recursion limit: `import sys; sys.setrecursionlimit(10000)`
- 

## 6.6 Interview Tips

- 💡 Start with the brute force recursive solution, then optimize
  - 💡 Draw the recursion tree — it shows the structure clearly
  - 💡 State your base case first — interviewers check this immediately
  - 💡 Ask: “Can elements be reused?” (changes `i` vs `i+1` in the loop)
  - 💡 Ask: “Are there duplicates in input?” (need to sort + skip)
  - 💡 Backtracking = modified DFS on a decision tree
- 

## 6.7 Practice Problems

| # | Problem                  | Type                    | Difficulty |
|---|--------------------------|-------------------------|------------|
| 1 | Subsets (LC #78)         | Backtracking            | Medium     |
| 2 | Permutations (LC #46)    | Backtracking            | Medium     |
| 3 | Combination Sum (LC #39) | Backtracking            | Medium     |
| 4 | Word Search (LC #79)     | Grid Backtracking       | Medium     |
| 5 | N-Queens (LC #51)        | Constraint Backtracking | Hard       |

---



## Revision Summary

- ✓ Recursion = function calls itself with smaller input + base case
- ✓ Always define BASE CASE first, then RECURSIVE CASE
- ✓ Call stack uses  $O(n)$  space – watch for stack overflow
- ✓ Backtracking = recursion + choose/explore/un-choose
- ✓ Template: choose → recurse → undo choice
- ✓ Subsets: include/exclude each element
- ✓ Permutations: try each remaining element at each position
- ✓ Always copy the path before adding to result
- ✓ Sort input to handle duplicates efficiently

---

*Next: Module 7 — Sorting & Searching →*

---

# Module 7: Sorting & Searching

## 7.1 Sorting Algorithms Overview

### Real-Life Analogy

**Sorting a deck of cards:** Different strategies exist — you can pick the smallest card each time (selection sort), insert each card into its correct position (insertion sort), or split the deck in half, sort each half, and merge them (merge sort).

### Comparison Table

| Algorithm      | Best          | Average       | Worst         | Space       | Stable? | Use Case                      |
|----------------|---------------|---------------|---------------|-------------|---------|-------------------------------|
| Bubble Sort    | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | ✓       | Educational only              |
| Selection Sort | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$      | ✗       | Educational only              |
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | ✓       | Small/nearly sorted           |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | ✓       | Guaranteed performance        |
| Quick Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | ✗       | General purpose (fastest avg) |
| Heap Sort      | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$      | ✗       | In-place, guaranteed          |
| Counting Sort  | $O(n+k)$      | $O(n+k)$      | $O(n+k)$      | $O(k)$      | ✓       | Small range integers          |
| Python Timsort | $O(n)$        | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | ✓       | Built-in <code>.sort()</code> |

**Stable** = equal elements maintain their original relative order.

## 7.2 Must-Know Sorting Algorithms

### Merge Sort — Divide & Conquer (Guaranteed $O(n \log n)$ )

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
```

```

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

```

# Time:  $O(n \log n)$  always | Space:  $O(n)$

## Quick Sort — Partition-Based (Fastest Average)

```

def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        pivot_idx = partition(arr, low, high)
        quick_sort(arr, low, pivot_idx - 1)
        quick_sort(arr, pivot_idx + 1, high)

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

```

# Time:  $O(n \log n)$  avg,  $O(n^2)$  worst | Space:  $O(\log n)$  – call stack

# Tip: Use random pivot to avoid worst case

## Counting Sort — $O(n)$ for Small Range Integers

```
def counting_sort(arr):
    if not arr:
        return arr
    min_val, max_val = min(arr), max(arr)
    count = [0] * (max_val - min_val + 1)

    for num in arr:
        count[num - min_val] += 1

    result = []
    for i, c in enumerate(count):
        result.extend([i + min_val] * c)
    return result

# Time:  $O(n + k)$  where  $k = \text{range}$  | Space:  $O(k)$ 
```

## Python Built-in Sorting

```
# .sort() – in-place, modifies original list
arr = [3, 1, 4, 1, 5]
arr.sort()                # [1, 1, 3, 4, 5]
arr.sort(reverse=True)    # [5, 4, 3, 1, 1]
arr.sort(key=lambda x: -x) # Sort by custom key

# sorted() – returns new list, original unchanged
new_arr = sorted(arr)
new_arr = sorted(arr, key=lambda x: x[1]) # Sort by 2nd element

# Sort by multiple criteria
students = [("Alice", 90), ("Bob", 85), ("Charlie", 90)]
students.sort(key=lambda x: (-x[1], x[0])) # By grade desc, then name asc

# Python uses Timsort:  $O(n \log n)$ , stable, hybrid of merge + insertion
```

---

## 7.3 Searching Algorithms

### Binary Search — $O(\log n)$ on Sorted Arrays

```
def binary_search(arr, target):
    """Standard binary search – returns index or -1."""
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2 # Avoids overflow
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# Time:  $O(\log n)$  | Space:  $O(1)$ 
```

### Binary Search Variants

```
def find_first(arr, target):
    """Find FIRST occurrence of target (leftmost)."""
    left, right = 0, len(arr) - 1
    result = -1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            result = mid
            right = mid - 1 # Keep searching left
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return result

def find_last(arr, target):
    """Find LAST occurrence of target (rightmost)."""
    left, right = 0, len(arr) - 1
    result = -1
    while left <= right:
        mid = left + (right - left) // 2
```

```

        if arr[mid] == target:
            result = mid
            left = mid + 1      # Keep searching right
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return result

def search_insert_position(arr, target):
    """Find index to insert target (bisect_left)."""
    left, right = 0, len(arr)
    while left < right:
        mid = left + (right - left) // 2
        if arr[mid] < target:
            left = mid + 1
        else:
            right = mid
    return left

# Python's bisect module
import bisect
idx = bisect.bisect_left(arr, target)    # First position >= target
idx = bisect.bisect_right(arr, target)   # First position > target
bisect.insort(arr, target)                # Insert maintaining order

```

## Binary Search on Answer Space

```

def min_eating_speed(piles, h):
    """Koko Eating Bananas (LC #875) – Binary search on answer."""
    def can_finish(speed):
        hours = sum((p + speed - 1) // speed for p in piles)
        return hours <= h

    left, right = 1, max(piles)
    while left < right:
        mid = left + (right - left) // 2
        if can_finish(mid):
            right = mid          # Try smaller speed
        else:
            left = mid + 1       # Need faster speed
    return left

# Time: O(n log m) where m = max(piles)

```

---

## 7.4 Pattern Recognition Tips

- 🔍 “Sort then solve” → Sort first, then two pointers / binary search
  - 🔍 “Kth largest/smallest” → Quick Select  $O(n)$  or Heap  $O(n \log k)$
  - 🔍 “Search in sorted array” → Binary Search
  - 🔍 “Minimum speed/capacity” → Binary Search on Answer
  - 🔍 “Search in rotated array” → Modified Binary Search
  - 🔍 “Find peak element” → Binary Search
  - 🔍 “Merge intervals” → Sort by start, then merge
- 

## 7.5 Common Mistakes

- ❌ Off-by-one in binary search: `left <= right` vs `left < right`
  - ❌ Integer overflow in `(left + right) / 2` — use `left + (right - left) // 2`
  - ❌ Not checking if array is sorted before applying binary search
  - ❌ Infinite loop when `left = mid` instead of `left = mid + 1`
  - ❌ Using  $O(n^2)$  sort when  $O(n \log n)$  is needed for large inputs
- 

## 7.6 Interview Tips

- 💡 Always use Python’s built-in `sort()` / `sorted()` unless asked to implement
  - 💡 Binary search is tested in 30%+ of coding interviews
  - 💡 Binary search on answer is a key advanced technique — learn it well
  - 💡 Know when sorting helps: It’s free to sort  $O(n \log n)$  if your algorithm is already  $O(n \log n)$
  - 💡 Quick Select for finding kth element —  $O(n)$  average
- 

## 7.7 Practice Problems

| # | Problem                             | Technique        | Difficulty |
|---|-------------------------------------|------------------|------------|
| 1 | Binary Search (LC #704)             | Standard BS      | Easy       |
| 2 | Search Insert Position (LC #35)     | BS variant       | Easy       |
| 3 | Search in Rotated Array (LC #33)    | Modified BS      | Medium     |
| 4 | Find Peak Element (LC #162)         | BS on unsorted   | Medium     |
| 5 | Median of Two Sorted Arrays (LC #4) | BS on partitions | Hard       |



## Solution: Search in Rotated Sorted Array (LC #33)

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid

        # Left half is sorted
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        # Right half is sorted
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1

# Time: O(log n) | Space: O(1)
```



## Revision Summary

- ✓ Merge Sort:  $O(n \log n)$  guaranteed, stable,  $O(n)$  space
- ✓ Quick Sort:  $O(n \log n)$  avg, in-place, use random pivot
- ✓ Python `.sort()` uses Timsort – always  $O(n \log n)$ , stable
- ✓ Binary Search:  $O(\log n)$  on sorted data, watch bounds carefully
- ✓ Binary Search on Answer: when searching for min/max valid value
- ✓ `bisect` module for Python binary search
- ✓ Stable sort preserves relative order of equal elements
- ✓ Know when to use `sort()` + two pointers vs hashmap

Next: Module 8 — Trees →

# Module 8: Trees (Binary Trees, BST, AVL)

## 8.1 What is a Tree?

**Definition:** A tree is a hierarchical data structure consisting of **nodes** connected by **edges**, with a single **root** node at the top and **no cycles**.

### Real-Life Analogy

A **family tree**: One ancestor (root) at the top, children branching below, grandchildren below that. Each person has exactly one parent (except the root), and there are no circular relationships.

### Tree Terminology

| Term         | Definition                     |
|--------------|--------------------------------|
| Root         | Top node (no parent)           |
| Leaf         | Node with no children          |
| Parent/Child | Direct connection              |
| Depth        | Distance from root to node     |
| Height       | Longest path from node to leaf |
| Subtree      | A node and all its descendants |
| Level        | All nodes at the same depth    |

### Binary Tree Node

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

## 8.2 Tree Traversals (The Foundation)

### DFS Traversals

```
# INORDER: Left → Root → Right (gives sorted order for BST)
```

```

def inorder(root):
    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# PREORDER: Root → Left → Right (copy/serialize a tree)
def preorder(root):
    if not root:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)

# POSTORDER: Left → Right → Root (delete tree, evaluate expression)
def postorder(root):
    if not root:
        return []
    return postorder(root.left) + postorder(root.right) + [root.val]

# Iterative Inorder (uses stack)
def inorder_iterative(root):
    result, stack = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result

```

## BFS Traversal (Level Order)

```

from collections import deque

def level_order(root):
    """BFS level-order traversal – returns list of levels."""
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level = []
        for _ in range(len(queue)):

```

```
        node = queue.popleft()
        level.append(node.val)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
        result.append(level)
```

```
    return result
```

```
# Time: O(n) | Space: O(n) – for all traversals
```

---

## 8.3 Essential Tree Problems

### Max Depth / Height

```
def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

```
# Time: O(n) | Space: O(h) where h = height
```

### Check if Balanced

```
def is_balanced(root):
    def check(node):
        if not node:
            return 0
        left = check(node.left)
        right = check(node.right)
        if left == -1 or right == -1 or abs(left - right) > 1:
            return -1
        return 1 + max(left, right)
    return check(root) != -1
```

```
# Time: O(n) | Space: O(h)
```

### Lowest Common Ancestor (LCA)

```
def lca(root, p, q):
```

```

"""Find LCA of nodes p and q in a binary tree."""
if not root or root == p or root == q:
    return root
left = lca(root.left, p, q)
right = lca(root.right, p, q)
if left and right:
    return root          # p and q on different sides
return left or right     # Both on same side
# Time: O(n) | Space: O(h)

```

## Diameter of Binary Tree

```

def diameter(root):
    result = [0]

    def depth(node):
        if not node:
            return 0
        left = depth(node.left)
        right = depth(node.right)
        result[0] = max(result[0], left + right)
        return 1 + max(left, right)

    depth(root)
    return result[0]
# Time: O(n) | Space: O(h)

```

## 8.4 Binary Search Tree (BST)

**Property:** For every node, left subtree values < node < right subtree values.

### BST Operations

```

class BST:
    def __init__(self):
        self.root = None

    def insert(self, val):
        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if not node:

```

```

        return TreeNode(val)
    if val < node.val:
        node.left = self._insert(node.left, val)
    elif val > node.val:
        node.right = self._insert(node.right, val)
    return node

def search(self, val):
    return self._search(self.root, val)

def _search(self, node, val):
    if not node or node.val == val:
        return node
    if val < node.val:
        return self._search(node.left, val)
    return self._search(node.right, val)

def delete(self, val):
    self.root = self._delete(self.root, val)

def _delete(self, node, val):
    if not node:
        return None
    if val < node.val:
        node.left = self._delete(node.left, val)
    elif val > node.val:
        node.right = self._delete(node.right, val)
    else:
        # Node found
        if not node.left:
            return node.right
        if not node.right:
            return node.left
        # Two children: replace with inorder successor
        successor = node.right
        while successor.left:
            successor = successor.left
        node.val = successor.val
        node.right = self._delete(node.right, successor.val)
    return node

def inorder(self):
    """Returns sorted elements."""
    result = []
    self._inorder(self.root, result)
    return result

def _inorder(self, node, result):

```

```
if node:
    self._inorder(node.left, result)
    result.append(node.val)
    self._inorder(node.right, result)
```

## BST Complexity

| Operation | Average     | Worst (skewed) |
|-----------|-------------|----------------|
| Search    | $O(\log n)$ | $O(n)$         |
| Insert    | $O(\log n)$ | $O(n)$         |
| Delete    | $O(\log n)$ | $O(n)$         |
| Inorder   | $O(n)$      | $O(n)$         |

## Validate BST

```
def is_valid_bst(root, lo=float('-inf'), hi=float('inf')):
    if not root:
        return True
    if root.val <= lo or root.val >= hi:
        return False
    return (is_valid_bst(root.left, lo, root.val) and
            is_valid_bst(root.right, root.val, hi))
# Time: O(n) | Space: O(h)
```

## 8.5 Pattern Recognition Tips

- 🔍 **“Level by level”** → BFS with queue
- 🔍 **“Height/depth”** → DFS recursion
- 🔍 **“Path sum”** → DFS with running sum
- 🔍 **“Serialize/deserialize”** → Preorder traversal
- 🔍 **“Sorted order from BST”** → Inorder traversal
- 🔍 **“Kth smallest in BST”** → Inorder traversal, stop at k
- 🔍 **“LCA”** → Recursion: check left and right subtrees
- 🔍 **“Symmetric / Mirror”** → Compare left.left with right.right

## 8.6 Common Mistakes

- ✗ Forgetting base case `if not root: return` — most tree bugs
  - ✗ Confusing depth and height — depth is from root, height is from leaf
  - ✗ Not passing bounds in BST validation — using just parent check is wrong
  - ✗ Using global variables — use `result = [0]` or pass accumulator
  - ✗ Not considering skewed trees — BST can degrade to  $O(n)$  linked list
- 

## 8.7 Interview Tips

- 💡 Most tree problems are solved with DFS recursion
  - 💡 Think: What info do I need from left/right subtrees?
  - 💡 BFS for level-related problems, DFS for path/height problems
  - 💡 BST inorder = sorted — use this property frequently
  - 💡 Draw the tree before coding — label the traversal order
  - 💡 Tree problems often have elegant  $O(n)$  solutions — don't overthink
- 

## 8.8 Practice Problems

| # | Problem                             | Type         | Difficulty |
|---|-------------------------------------|--------------|------------|
| 1 | Maximum Depth (LC #104)             | DFS          | Easy       |
| 2 | Invert Binary Tree (LC #226)        | DFS          | Easy       |
| 3 | Validate BST (LC #98)               | DFS + bounds | Medium     |
| 4 | Binary Tree Level Order (LC #102)   | BFS          | Medium     |
| 5 | Serialize and Deserialize (LC #297) | Preorder     | Hard       |

---



## Revision Summary

- ✓ Tree = hierarchical, no cycles, one root
  - ✓ Binary tree: at most 2 children per node
  - ✓ Traversals: Inorder (L-R-R), Preorder (R-L-R), Postorder (L-R-R), Level Order (BFS)
  - ✓ BST property: `left < node < right`
  - ✓ BST inorder = sorted order
  - ✓ Height of balanced tree =  $O(\log n)$
  - ✓ Most tree problems: recursive DFS with base case check
  - ✓ LCA: if found in both subtrees, current node is LCA
-



*Next: Module 9 — Heaps & Priority Queues →*

---

# Module 9: Heaps & Priority Queues

## 9.1 What is a Heap?

**Definition:** A heap is a **complete binary tree** stored as an array where every parent is smaller (min-heap) or larger (max-heap) than its children.

### Real-Life Analogy

**Hospital emergency room:** Patients aren't served first-come-first-served. The most critical patient (highest priority) is treated first, regardless of arrival time. That's a priority queue backed by a heap.

### Heap Properties

- **Min-Heap:**  $\text{Parent} \leq \text{Children}$  (root = minimum)
- **Max-Heap:**  $\text{Parent} \geq \text{Children}$  (root = maximum)
- **Complete tree:** All levels full except possibly the last (filled left to right)
- **Array representation:** For node at index  $i$ :
  - Parent:  $(i - 1) // 2$
  - Left child:  $2 * i + 1$
  - Right child:  $2 * i + 2$

### Operations Complexity

| Operation             | Time        |
|-----------------------|-------------|
| Insert (push)         | $O(\log n)$ |
| Extract min/max (pop) | $O(\log n)$ |
| Peek min/max          | $O(1)$      |
| Build heap from array | $O(n)$      |
| Search                | $O(n)$      |

## 9.2 Python heapq Module (Min-Heap)

```
import heapq

# Python heapq is a MIN-HEAP
heap = []

heapq.heappush(heap, 5)      # Push:  $O(\log n)$ 
```

```

heapq.heappush(heap, 2)
heapq.heappush(heap, 8)
heapq.heappush(heap, 1)

print(heap[0])           # Peek min: O(1) → 1
min_val = heapq.heappop(heap) # Pop min: O(log n) → 1

# Build heap from existing list: O(n)
arr = [5, 2, 8, 1, 9]
heapq.heapify(arr)       # In-place, O(n)

# Push and pop in one step (more efficient)
heapq.heappushpop(heap, 3) # Push 3, then pop min
heapq.heapreplace(heap, 3) # Pop min, then push 3

# Get k smallest/largest
heapq.nsmallest(3, arr)   # [1, 2, 5]
heapq.nlargest(3, arr)    # [9, 8, 5]

# ⚠️ MAX-HEAP trick: negate values
max_heap = []
heapq.heappush(max_heap, -5) # Store negated
heapq.heappush(max_heap, -2)
max_val = -heapq.heappop(max_heap) # Negate back → 5

# Custom comparison: use tuples (priority, value)
tasks = []
heapq.heappush(tasks, (1, "Low priority"))
heapq.heappush(tasks, (3, "High priority"))
heapq.heappush(tasks, (2, "Medium priority"))
priority, task = heapq.heappop(tasks) # (1, "Low priority")

```

## 9.3 Heap Implementation from Scratch

```

class MinHeap:
    def __init__(self):
        self.heap = []

    def push(self, val):
        self.heap.append(val)
        self._sift_up(len(self.heap) - 1)

    def pop(self):
        if not self.heap:

```

```

        raise IndexError("Heap is empty")
    # Swap root with last, remove last, sift down
    self.heap[0], self.heap[-1] = self.heap[-1], self.heap[0]
    min_val = self.heap.pop()
    if self.heap:
        self._sift_down(0)
    return min_val

def peek(self):
    return self.heap[0] if self.heap else None

def _sift_up(self, idx):
    parent = (idx - 1) // 2
    while idx > 0 and self.heap[idx] < self.heap[parent]:
        self.heap[idx], self.heap[parent] = self.heap[parent], self.heap[idx]
        idx = parent
        parent = (idx - 1) // 2

def _sift_down(self, idx):
    n = len(self.heap)
    while True:
        smallest = idx
        left = 2 * idx + 1
        right = 2 * idx + 2
        if left < n and self.heap[left] < self.heap[smallest]:
            smallest = left
        if right < n and self.heap[right] < self.heap[smallest]:
            smallest = right
        if smallest == idx:
            break
        self.heap[idx], self.heap[smallest] = self.heap[smallest], self.heap[idx]
        idx = smallest

```

## 9.4 Classic Heap Problems

### Kth Largest Element (LC #215)

```

def find_kth_largest(nums, k):
    """Use min-heap of size k."""
    heap = nums[:k]
    heapq.heapify(heap)

    for num in nums[k:]:
        if num > heap[0]:

```

```

        heapq.heappreplaced(heap, num)

    return heap[0]

# Time: O(n log k) | Space: O(k)

```

## Merge K Sorted Lists (LC #23)

```

def merge_k_lists(lists):
    """Merge k sorted linked lists using heap."""
    heap = []
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst.val, i, lst))

    dummy = curr = ListNode(0)
    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))

    return dummy.next

# Time: O(N log k) where N = total nodes | Space: O(k)

```

## Find Median from Data Stream (LC #295)

```

class MedianFinder:
    """Two heaps: max-heap for lower half, min-heap for upper half."""
    def __init__(self):
        self.small = [] # Max-heap (negated) - lower half
        self.large = [] # Min-heap - upper half

    def addNum(self, num):
        heapq.heappush(self.small, -num)

        # Ensure max of small <= min of large
        if self.small and self.large and -self.small[0] > self.large[0]:
            val = -heapq.heappop(self.small)
            heapq.heappush(self.large, val)

```

```
# Balance sizes (small can have at most 1 extra)
if len(self.small) > len(self.large) + 1:
    val = -heapq.heappop(self.small)
    heapq.heappush(self.large, val)
elif len(self.large) > len(self.small):
    val = heapq.heappop(self.large)
    heapq.heappush(self.small, -val)

def findMedian(self):
    if len(self.small) > len(self.large):
        return -self.small[0]
    return (-self.small[0] + self.large[0]) / 2

# addNum: O(log n) | findMedian: O(1)
```

---

## 9.5 Pattern Recognition Tips

- 🔍 “Kth largest/smallest” → Min/Max heap of size k
- 🔍 “Merge k sorted” → Min-heap with one element from each
- 🔍 “Top k frequent” → Counter + Heap or Bucket Sort
- 🔍 “Running median” → Two heaps (max-heap + min-heap)
- 🔍 “Schedule tasks by priority” → Priority Queue
- 🔍 “Closest k points” → Max-heap of size k

---

## 9.6 Common Mistakes

- ❌ **Forgetting Python heapq is min-heap** — negate for max-heap
- ❌ **Not using tuples** for custom comparison — heapq can’t use custom comparators
- ❌ **Modifying heap elements directly** — invalidates heap property
- ❌ **Using heap when sorting works** — if you need all elements sorted, just sort
- ❌ **Not considering heapq.nsmallest/nlargest** — cleaner for simple cases

---

## 9.7 Interview Tips

- 💡 **Default to heapq** in Python interviews — clean and efficient
  - 💡 “Kth” in the problem → think Heap immediately
  - 💡 **Two-heap pattern** for median problems is a FAANG classic
  - 💡 **Heap vs sorting**: Heap is better when  $k \ll n$  ( $O(n \log k)$  vs  $O(n \log n)$ )
  - 💡 Know that `heapify` is  $O(n)$ , not  $O(n \log n)$
-

## 9.8 Practice Problems

| # | Problem                                | Pattern             | Difficulty |
|---|----------------------------------------|---------------------|------------|
| 1 | Kth Largest Element (LC #215)          | Min-heap size k     | Medium     |
| 2 | Top K Frequent Elements (LC #347)      | Counter + Heap      | Medium     |
| 3 | Merge K Sorted Lists (LC #23)          | Multi-way merge     | Hard       |
| 4 | Find Median from Data Stream (LC #295) | Two heaps           | Hard       |
| 5 | Task Scheduler (LC #621)               | Max-heap + cooldown | Medium     |



### Revision Summary

- ✓ Heap = complete binary tree stored as array
- ✓ Min-Heap:  $\text{parent} \leq \text{children}$ , root = minimum
- ✓ Python heapq: min-heap, negate for max-heap
- ✓ Push/Pop:  $O(\log n)$ , Peek:  $O(1)$ , Heapify:  $O(n)$
- ✓ Kth element  $\rightarrow$  heap of size k
- ✓ Merge k sorted  $\rightarrow$  min-heap with one from each
- ✓ Running median  $\rightarrow$  two heaps (small max-heap + large min-heap)
- ✓ Use tuples (priority, value) for custom ordering

*Next: Module 10 — Graphs  $\rightarrow$*

# Module 10: Graphs

## 10.1 What is a Graph?

**Definition:** A graph is a collection of **vertices (nodes)** connected by **edges**. Unlike trees, graphs can have **cycles**, **multiple paths**, and **no root**.

### Real-Life Analogy

**City road network:** Cities are nodes, roads are edges. Some roads are one-way (directed), some have tolls (weighted). You might want the shortest route (Dijkstra) or to visit all cities (traversal).

### Graph Types

| Type          | Description          | Example            |
|---------------|----------------------|--------------------|
| Undirected    | Edges go both ways   | Facebook friends   |
| Directed      | Edges have direction | Twitter follows    |
| Weighted      | Edges have costs     | Road distances     |
| Unweighted    | All edges equal      | Social connections |
| Cyclic        | Contains cycles      | Road network       |
| Acyclic (DAG) | No cycles, directed  | Task dependencies  |

## 10.2 Graph Representation

```
# Method 1: Adjacency List (MOST COMMON – use this)
from collections import defaultdict

graph = defaultdict(list)

# Undirected
graph[0].append(1)
graph[1].append(0)

# Directed
graph["A"].append("B")

# From edge list
edges = [[0,1], [0,2], [1,3], [2,3]]
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
```



```
graph[v].append(u) # Remove for directed

# Method 2: Adjacency Matrix (for dense graphs)
n = 4
matrix = [[0] * n for _ in range(n)]
matrix[0][1] = 1 # Edge from 0 to 1
matrix[1][0] = 1 # Undirected

# Method 3: Edge List (simple but slow for lookups)
edges = [(0, 1, 5), (0, 2, 3)] # (from, to, weight)
```

## When to Use What

| Representation   | Space    | Check edge         | Get neighbors | Best for      |
|------------------|----------|--------------------|---------------|---------------|
| Adjacency List   | $O(V+E)$ | $O(\text{degree})$ | $O(1)$        | Sparse graphs |
| Adjacency Matrix | $O(V^2)$ | $O(1)$             | $O(V)$        | Dense graphs  |

# 10.3 Graph Traversals

## BFS — Breadth-First Search

```
from collections import deque

def bfs(graph, start):
    """Visit all nodes level by level."""
    visited = {start}
    queue = deque([start])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return result

# Time:  $O(V + E)$  | Space:  $O(V)$ 
# Use for: Shortest path (unweighted), level-order, nearest neighbor
```

## DFS — Depth-First Search

```
# Iterative DFS
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    result = []

    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        result.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                stack.append(neighbor)

    return result

# Recursive DFS
def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    result = [node]

    for neighbor in graph[node]:
        if neighbor not in visited:
            result.extend(dfs_recursive(graph, neighbor, visited))

    return result

# Time: O(V + E) | Space: O(V)
# Use for: Path finding, cycle detection, topological sort, connected components
```

---

## 10.4 Essential Graph Algorithms

### Shortest Path — BFS (Unweighted)

```
def shortest_path(graph, start, end):
    """Find shortest path in unweighted graph."""
```

```

queue = deque([(start, [start])])
visited = {start}

while queue:
    node, path = queue.popleft()
    if node == end:
        return path

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append((neighbor, path + [neighbor]))

return [] # No path

# Time: O(V + E) | Space: O(V)

```

## Dijkstra's Algorithm (Weighted, Non-negative)

```

import heapq

def dijkstra(graph, start):
    """Shortest distances from start to all nodes.
    graph[u] = [(v, weight), ...]
    """
    distances = {start: 0}
    heap = [(0, start)] # (distance, node)

    while heap:
        dist, node = heapq.heappop(heap)

        if dist > distances.get(node, float('inf')):
            continue # Skip outdated entry

        for neighbor, weight in graph[node]:
            new_dist = dist + weight
            if new_dist < distances.get(neighbor, float('inf')):
                distances[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))

    return distances

# Time: O((V + E) log V) | Space: O(V)

```

# Cycle Detection

```
# Undirected graph – Union Find or DFS
def has_cycle_undirected(graph, n):
    visited = set()

    def dfs(node, parent):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                if dfs(neighbor, node):
                    return True
            elif neighbor != parent:
                return True # Found cycle
        return False

    for i in range(n):
        if i not in visited:
            if dfs(i, -1):
                return True
    return False

# Directed graph – DFS with 3 colors
def has_cycle_directed(graph, n):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * n

    def dfs(node):
        color[node] = GRAY
        for neighbor in graph[node]:
            if color[neighbor] == GRAY:
                return True # Back edge = cycle
            if color[neighbor] == WHITE and dfs(neighbor):
                return True
        color[node] = BLACK
        return False

    return any(color[i] == WHITE and dfs(i) for i in range(n))
```

# Topological Sort (DAG only)

```
def topological_sort(graph, n):
    """Kahn's algorithm – BFS-based topological sort."""
    in_degree = [0] * n
```

```

for u in graph:
    for v in graph[u]:
        in_degree[v] += 1

queue = deque([i for i in range(n) if in_degree[i] == 0])
order = []

while queue:
    node = queue.popleft()
    order.append(node)
    for neighbor in graph[node]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            queue.append(neighbor)

return order if len(order) == n else [] # Empty = cycle exists

# Time: O(V + E) | Space: O(V)

```

## Number of Islands (LC #200) — Grid BFS/DFS

```

def num_islands(grid):
    """Count connected components of '1's in a grid."""
    if not grid:
        return 0

    rows, cols = len(grid), len(grid[0])
    count = 0

    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1':
            return
        grid[r][c] = '0' # Mark visited
        dfs(r+1, c)
        dfs(r-1, c)
        dfs(r, c+1)
        dfs(r, c-1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)

    return count

```

```
# Time: O(rows × cols) | Space: O(rows × cols) – recursion stack
```

## Union-Find (Disjoint Set)

```
class UnionFind:
    """Optimized with path compression + union by rank."""
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # Path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False # Already connected
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        self.components -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)

# find/union: nearly O(1) amortized (inverse Ackermann)
```

## 10.5 Pattern Recognition Tips

-  “Shortest path (unweighted)” → BFS
-  “Shortest path (weighted)” → Dijkstra
-  “Connected components / islands” → DFS/BFS or Union-Find
-  “Cycle detection” → DFS with parent (undirected) or colors (directed)
-  “Task ordering / prerequisites” → Topological Sort

- 🔍 “Grid traversal” → Treat grid as graph, 4-directional DFS/BFS
  - 🔍 “Is graph bipartite?” → BFS/DFS with 2-coloring
- 

## 10.6 Common Mistakes

- ❌ **Forgetting visited set** → infinite loops in cyclic graphs
  - ❌ **BFS without a queue** — don’t use `list.pop(0)`, use deque
  - ❌ **Using BFS for weighted shortest path** — use Dijkstra instead
  - ❌ **Not handling disconnected graphs** — loop over all nodes
  - ❌ **Grid bounds checking** — always validate row/col before accessing
- 

## 10.7 Practice Problems

| # | Problem                      | Algorithm        | Difficulty |
|---|------------------------------|------------------|------------|
| 1 | Number of Islands (LC #200)  | DFS/BFS grid     | Medium     |
| 2 | Clone Graph (LC #133)        | BFS + HashMap    | Medium     |
| 3 | Course Schedule (LC #207)    | Topological Sort | Medium     |
| 4 | Word Ladder (LC #127)        | BFS              | Hard       |
| 5 | Network Delay Time (LC #743) | Dijkstra         | Medium     |

---

## Revision Summary

- ✅ Graph = vertices + edges (directed/undirected, weighted/unweighted)
  - ✅ Adjacency list for sparse graphs, matrix for dense
  - ✅ BFS: level-by-level, shortest path (unweighted), uses queue
  - ✅ DFS: go deep first, uses stack/recursion
  - ✅ Dijkstra: shortest path with non-negative weights, uses heap
  - ✅ Topological Sort: ordering for DAGs (Kahn's BFS or DFS)
  - ✅ Union-Find: connected components, cycle detection in undirected
  - ✅ Grid = implicit graph, 4 neighbors per cell
- 

*Next: Module 11 — Dynamic Programming →*

---

# Module 11: Dynamic Programming

## 11.1 What is Dynamic Programming?

**Definition:** DP solves complex problems by breaking them into **overlapping subproblems**, solving each subproblem **once**, and storing results to avoid redundant computation.

### Real-Life Analogy

**Writing an exam cheat sheet:** Instead of re-deriving every formula during the test, you **pre-compute** and write them down. Next time you need a formula, you just look it up instead of recalculating.

### When to Use DP

A problem has DP potential if it has:

1. **Optimal Substructure:** Optimal solution is built from optimal solutions of subproblems
2. **Overlapping Subproblems:** Same subproblem is solved multiple times

### Two Approaches

| Approach                      | Direction              | Method            | Pros                            |
|-------------------------------|------------------------|-------------------|---------------------------------|
| <b>Top-Down (Memoization)</b> | Start from big → small | Recursion + cache | Easier to write                 |
| <b>Bottom-Up (Tabulation)</b> | Start from small → big | Iterative + table | Better space, no stack overflow |

```
# Example: Fibonacci

# ❌ Naive Recursion – O(2^n) TIME!
def fib_naive(n):
    if n <= 1: return n
    return fib_naive(n-1) + fib_naive(n-2)

# ✅ Top-Down (Memoization) – O(n) time, O(n) space
from functools import lru_cache

@lru_cache(maxsize=None)
def fib_memo(n):
    if n <= 1: return n
```



```

    return fib_memo(n-1) + fib_memo(n-2)

# ✅ Bottom-Up (Tabulation) – O(n) time, O(n) space
def fib_tab(n):
    if n <= 1: return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# ✅ Space-Optimized – O(n) time, O(1) space
def fib_optimal(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

```

## 11.2 The DP Framework (5 Steps)

For EVERY DP problem, follow these steps:

1. DEFINE the subproblem: What does `dp[i]` represent?
2. WRITE the recurrence relation: `dp[i] = f(dp[j], ...)`
3. IDENTIFY the base cases: `dp[0] = ?`, `dp[1] = ?`
4. DETERMINE the iteration order: Left→right? Bottom→up?
5. OPTIMIZE space if possible

## 11.3 DP Pattern 1: 1D DP

### Climbing Stairs (LC #70)

```

def climb_stairs(n):
    """Number of ways to climb n stairs (1 or 2 steps at a time).
    dp[i] = number of ways to reach step i
    dp[i] = dp[i-1] + dp[i-2]
    """
    if n <= 2: return n
    a, b = 1, 2
    for _ in range(3, n + 1):
        a, b = b, a + b
    return b

```

```
# Time: O(n) | Space: O(1)
```

## House Robber (LC #198)

```
def rob(nums):
    """Max money robbing non-adjacent houses.
    dp[i] = max money robbing from house 0 to i
    dp[i] = max(dp[i-1], dp[i-2] + nums[i])
    """
    if not nums: return 0
    if len(nums) == 1: return nums[0]

    prev2, prev1 = 0, 0
    for num in nums:
        curr = max(prev1, prev2 + num)
        prev2, prev1 = prev1, curr
    return prev1
```

```
# Time: O(n) | Space: O(1)
```

## Coin Change (LC #322)

```
def coin_change(coins, amount):
    """Minimum coins to make amount.
    dp[i] = min coins needed for amount i
    dp[i] = min(dp[i - coin] + 1) for all coins
    """
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i and dp[i - coin] != float('inf'):
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount] if dp[amount] != float('inf') else -1
```

```
# Time: O(amount × len(coins)) | Space: O(amount)
```

## Longest Increasing Subsequence (LC #300)

```
def length_of_lis(nums):
    """Length of longest increasing subsequence.
```

```

dp[i] = length of LIS ending at index i
dp[i] = max(dp[j] + 1) for all j < i where nums[j] < nums[i]
"""

if not nums: return 0
dp = [1] * len(nums)

for i in range(1, len(nums)):
    for j in range(i):
        if nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j] + 1)

return max(dp)
# Time: O(n²) | Space: O(n)
# Can be optimized to O(n log n) with binary search + patience sorting

import bisect
def length_of_lis_optimized(nums):
    tails = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
# Time: O(n log n) | Space: O(n)

```

## 11.4 DP Pattern 2: 2D DP

### Unique Paths (LC #62)

```

def unique_paths(m, n):
    """Paths from top-left to bottom-right in m×n grid.
    dp[i][j] = number of paths to reach (i, j)
    dp[i][j] = dp[i-1][j] + dp[i][j-1]
    """
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]

```

# Time:  $O(m \times n)$  | Space:  $O(m \times n)$ , optimizable to  $O(n)$

## Longest Common Subsequence (LC #1143)

```
def lcs(text1, text2):
    """Length of longest common subsequence.
    dp[i][j] = LCS of text1[:i] and text2[:j]
    """
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
```

# Time:  $O(m \times n)$  | Space:  $O(m \times n)$

## Edit Distance (LC #72)

```
def min_distance(word1, word2):
    """Minimum operations to convert word1 to word2.
    Operations: insert, delete, replace.
    dp[i][j] = edit distance between word1[:i] and word2[:j]
    """
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i-1] == word2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(
                    dp[i-1][j],      # Delete
                    dp[i][j-1],      # Insert
                    dp[i-1][j-1] + 1 # Replace
                )
```

```

        dp[i][j-1],      # Insert
        dp[i-1][j-1]     # Replace
    )

    return dp[m][n]
# Time: O(m×n) | Space: O(m×n)

```

## 11.5 DP Pattern 3: Knapsack

### 0/1 Knapsack

```

def knapsack_01(weights, values, capacity):
    """Max value with given capacity (each item used at most once).
    dp[i][w] = max value using items 0..i with capacity w
    """
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            dp[i][w] = dp[i-1][w] # Skip item i
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w],
                               dp[i-1][w - weights[i-1]] + values[i-1])

    return dp[n][capacity]

# Space-optimized (1D array)
def knapsack_01_optimized(weights, values, capacity):
    dp = [0] * (capacity + 1)
    for i in range(len(weights)):
        for w in range(capacity, weights[i] - 1, -1): # Reverse!
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
    return dp[capacity]
# Time: O(n × capacity) | Space: O(capacity)

```

### Unbounded Knapsack (Coin Change variant)

```

def unbounded_knapsack(weights, values, capacity):
    """Items can be used unlimited times."""
    dp = [0] * (capacity + 1)
    for w in range(1, capacity + 1):

```

```
for i in range(len(weights)):
    if weights[i] <= w:
        dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
return dp[capacity]
```

## 11.6 DP Pattern 4: Interval DP & Other Patterns

### Palindromic Substrings (LC #647)

```
def count_palindromes(s):
    """Count all palindromic substrings using expand around center."""
    count = 0
    for center in range(len(s)):
        # Odd length
        l, r = center, center
        while l >= 0 and r < len(s) and s[l] == s[r]:
            count += 1
            l -= 1
            r += 1
        # Even length
        l, r = center, center + 1
        while l >= 0 and r < len(s) and s[l] == s[r]:
            count += 1
            l -= 1
            r += 1
    return count
# Time: O(n^2) | Space: O(1)
```

## 11.7 Pattern Recognition Tips

- 🔍 “Minimum/maximum of something” → DP
- 🔍 “Number of ways to...” → DP
- 🔍 “Can you reach / is it possible” → DP (or greedy)
- 🔍 “Subsequence” → 1D or 2D DP
- 🔍 “Two strings comparison” → 2D DP (LCS, Edit Distance)
- 🔍 “Partition into subsets” → Knapsack variant
- 🔍 “Decision at each step: take or skip” → 0/1 Knapsack
- 🔍 “Unlimited usage” → Unbounded Knapsack

# DP Problem Categories Cheat Sheet

|                |                                                       |
|----------------|-------------------------------------------------------|
| 1D DP:         | Fibonacci, Climbing Stairs, House Robber, Coin Change |
| 2D DP:         | LCS, Edit Distance, Unique Paths                      |
| Knapsack:      | Subset Sum, Partition Equal Subset, Target Sum        |
| String DP:     | Palindrome, Word Break, Regular Expression            |
| Tree DP:       | House Robber III, Diameter, Max Path Sum              |
| Interval DP:   | Matrix Chain, Burst Balloons                          |
| State Machine: | Stock problems, with cooldown                         |
| Bitmask DP:    | Traveling Salesman, assignment problems               |

---

## 11.8 Common Mistakes

- ✗ **Jumping to DP without thinking** — try greedy or brute force first
  - ✗ **Wrong base case** — most bugs come from initialization
  - ✗ **Wrong iteration order** — bottom-up must build from smaller subproblems
  - ✗ **Not defining  $dp[i]$  clearly** — always write what  $dp[i]$  means
  - ✗ **Off-by-one in 2D DP** — use  $dp$  of size  $(m+1) \times (n+1)$  for strings
  - ✗ **Forgetting space optimization** — iterate in reverse for 1D knapsack
- 

## 11.9 Interview Tips

- 💡 **Start with recursion** → add memoization → convert to tabulation
  - 💡 **Always define  $dp[i]$  in words** before writing the recurrence
  - 💡 **Draw a small example** and fill the DP table by hand
  - 💡 **State your recurrence relation** clearly to the interviewer
  - 💡 DP interviews test **pattern recognition** — practice categorizing problems
  - 💡 **Space optimization** is a bonus that impresses interviewers
- 

## 11.10 Practice Problems

| # Problem                               | Pattern                    | Difficulty |
|-----------------------------------------|----------------------------|------------|
| 1 Climbing Stairs (LC #70)              | 1D DP                      | Easy       |
| 2 House Robber (LC #198)                | 1D DP                      | Medium     |
| 3 Coin Change (LC #322)                 | 1D DP / Unbounded Knapsack | Medium     |
| 4 Longest Common Subsequence (LC #1143) | 2D DP                      | Medium     |
| 5 Edit Distance (LC #72)                | 2D DP                      | Medium     |

---



## Revision Summary

- ✓ DP = overlapping subproblems + optimal substructure
  - ✓ Top-Down: recursion + cache (@lru\_cache)
  - ✓ Bottom-Up: iterative with dp table
  - ✓ 5-Step Framework: define dp[i], recurrence, base case, order, optimize
  - ✓ 1D DP: Fibonacci, House Robber, Coin Change
  - ✓ 2D DP: LCS, Edit Distance, Grid Paths
  - ✓ Knapsack: 0/1 (reverse loop) vs Unbounded (forward loop)
  - ✓ Always start with recursive solution, then optimize
  - ✓ Practice categorizing problems into DP patterns
- 

*Next: Module 12 — Greedy Algorithms →*

---



# Module 12: Greedy Algorithms

## 12.1 What is Greedy?

**Definition:** A greedy algorithm makes the **locally optimal choice** at each step, hoping to reach a **globally optimal solution**. Unlike DP, it never reconsiders past choices.

### Real-Life Analogy

**Making change with coins:** To give \$0.87 in change, you pick the **largest coin** that fits: 50¢ → 25¢ → 10¢ → 1¢ → 1¢. This greedy approach works for standard coin denominations (but not always for arbitrary ones).

### Greedy vs DP

| Feature     | Greedy                | DP                                  |
|-------------|-----------------------|-------------------------------------|
| Approach    | Local best choice     | Explore all options                 |
| Speed       | Usually $O(n \log n)$ | Usually $O(n^2)$ or $O(n \times m)$ |
| Correctness | Only when proven      | Always correct                      |
| Difficulty  | Hard to prove correct | Hard to implement                   |
| Examples    | Intervals, Huffman    | Knapsack, LCS                       |

### When Does Greedy Work?

Greedy works when:

1. **Greedy choice property:** A local optimum leads to a global optimum
2. **Optimal substructure:** Optimal solution contains optimal sub-solutions

 **You must prove (or intuit) that greedy works.** Wrong greedy = wrong answer.

## 12.2 Classic Greedy Problems

### Activity Selection / Meeting Rooms II

```
def min_meeting_rooms(intervals):  
    """Minimum number of meeting rooms needed.  
    Sort by start time, use min-heap to track end times.  
    """
```

```

import heapq
if not intervals:
    return 0

intervals.sort(key=lambda x: x[0])
heap = [intervals[0][1]] # End time of first meeting

for start, end in intervals[1:]:
    if start >= heap[0]:
        heapq.heappop(heap) # Reuse room
        heapq.heappush(heap, end)

return len(heap)

# Time: O(n log n) | Space: O(n)

```

## Non-overlapping Intervals (LC #435)

```

def erase_overlap_intervals(intervals):
    """Minimum intervals to remove for no overlap.
    Greedy: sort by end time, keep non-overlapping.
    """
    intervals.sort(key=lambda x: x[1])
    count = 0
    prev_end = float('-inf')

    for start, end in intervals:
        if start >= prev_end:
            prev_end = end # Keep this interval
        else:
            count += 1     # Remove this interval

    return count

# Time: O(n log n) | Space: O(1)

```

## Jump Game (LC #55)

```

def can_jump(nums):
    """Can you reach the last index?
    Greedy: track the farthest reachable position.
    """
    farthest = 0

```

```

for i in range(len(nums)):
    if i > farthest:
        return False    # Can't reach this index
    farthest = max(farthest, i + nums[i])
return True

```

# Time:  $O(n)$  | Space:  $O(1)$

## Jump Game II (LC #45)

```

def jump(nums):
    """Minimum jumps to reach last index."""
    jumps = 0
    current_end = 0
    farthest = 0

    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == current_end:
            jumps += 1
            current_end = farthest

    return jumps

```

# Time:  $O(n)$  | Space:  $O(1)$

## Gas Station (LC #134)

```

def can_complete_circuit(gas, cost):
    """Find starting station to complete circuit."""
    if sum(gas) < sum(cost):
        return -1    # Impossible

    tank = 0
    start = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            start = i + 1    # Restart from next station
            tank = 0

    return start

```

# Time:  $O(n)$  | Space:  $O(1)$

## Task Scheduler (LC #621)

```
def least_interval(tasks, n):  
    """Minimum intervals to execute all tasks with cooldown n."""  
    from collections import Counter  
    freq = Counter(tasks)  
    max_freq = max(freq.values())  
    max_count = sum(1 for v in freq.values() if v == max_freq)  
  
    # (max_freq - 1) full groups + last group with max_count tasks  
    result = (max_freq - 1) * (n + 1) + max_count  
    return max(result, len(tasks))
```

# Time:  $O(n)$  | Space:  $O(1)$  – at most 26 task types

## 12.3 Pattern Recognition Tips

- 🔍 “Minimum rooms/resources” → Sort + Greedy (or Heap)
- 🔍 “Maximum non-overlapping intervals” → Sort by end time
- 🔍 “Can you reach the end?” → Greedy forward scan
- 🔍 “Minimum jumps/steps” → BFS-like greedy
- 🔍 “Assign tasks/resources” → Sort by some criteria + greedy assign
- 🔍 “Merge intervals” → Sort by start time
- 🔍 “Partition labels” → Track last occurrence

## Greedy Proof Strategies

1. **Exchange argument:** Show swapping any non-greedy choice for the greedy choice doesn't make things worse
2. **Stay ahead:** Show greedy is always at least as good as any other algorithm at every step

## 12.4 Common Mistakes

- ❌ **Assuming greedy works without proof** — it often doesn't (e.g., 0/1 knapsack)
- ❌ **Wrong sorting criteria** — sort by end time for interval scheduling
- ❌ **Not handling ties** in sorting
- ❌ **Confusing greedy with DP** — if you need to try all options, it's DP
- ❌ **Greedy for coin change** — fails for non-standard denominations

---

## 12.5 Interview Tips

- 💡 Try **greedy first** if the problem feels like it — it's simpler and faster
  - 💡 If greedy fails, **fall back to DP**
  - 💡 **Sort the input** — 80% of greedy problems require sorting first
  - 💡 **Explain WHY greedy works** — interviewers want to see your reasoning
  - 💡 Interval problems are the **most common** greedy category in interviews
- 

## 12.6 Practice Problems

| # | Problem                             | Type         | Difficulty |
|---|-------------------------------------|--------------|------------|
| 1 | Jump Game (LC #55)                  | Reachability | Medium     |
| 2 | Non-overlapping Intervals (LC #435) | Interval     | Medium     |
| 3 | Gas Station (LC #134)               | Circular     | Medium     |
| 4 | Task Scheduler (LC #621)            | Scheduling   | Medium     |
| 5 | Jump Game II (LC #45)               | Min steps    | Medium     |

---



## Revision Summary

- ✓ Greedy = local optimal choice → global optimal (when it works)
  - ✓ Must have: greedy choice property + optimal substructure
  - ✓ Sort first in 80% of greedy problems
  - ✓ Intervals: sort by end time for max non-overlapping
  - ✓ Jump Game: track farthest reachable position
  - ✓ If greedy fails → use DP
  - ✓ Always explain WHY greedy works in interviews
  - ✓ Common categories: intervals, scheduling, reachability
- 

*Next: Module 13 — Tries & Advanced Structures →*

---

# Module 13: Tries & Advanced Data Structures

---

## 13.1 Trie (Prefix Tree)

**Definition:** A trie is a tree-like structure where each node represents a character. Paths from root to nodes represent prefixes of stored strings. Enables  $O(L)$  lookup where  $L$  = word length.



### Real-Life Analogy

**Autocomplete on your phone:** As you type “app”, the system instantly finds “apple”, “application”, “app store” — it’s traversing a trie of all known words.

### Trie Implementation

```
class TrieNode:
    def __init__(self):
        self.children = {}      # char → TrieNode
        self.is_end = False    # Marks end of a word

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        """Insert a word – O(L)."""
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end = True

    def search(self, word):
        """Check if exact word exists – O(L)."""
        node = self._find_node(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        """Check if any word starts with prefix – O(L)."""
        return self._find_node(prefix) is not None
```

```

def _find_node(self, prefix):
    node = self.root
    for char in prefix:
        if char not in node.children:
            return None
        node = node.children[char]
    return node

# Usage
trie = Trie()
trie.insert("apple")
trie.insert("app")
trie.search("apple")      # True
trie.search("app")        # True
trie.search("ap")         # False
trie.starts_with("ap")    # True

```

## Trie Complexity

| Operation    | Time   | Space           |
|--------------|--------|-----------------|
| Insert       | $O(L)$ | $O(L)$          |
| Search       | $O(L)$ | $O(1)$          |
| Prefix check | $O(L)$ | $O(1)$          |
| Total space  | —      | $O(N \times L)$ |

$L$  = word length,  $N$  = number of words

## Word Search II (LC #212) — Trie + Backtracking

```

def find_words(board, words):
    """Find all words from dictionary in a grid."""
    trie = Trie()
    for word in words:
        trie.insert(word)

    rows, cols = len(board), len(board[0])
    result = set()

    def dfs(r, c, node, path):
        if node.is_end:
            result.add(path)
            node.is_end = False # Avoid duplicates

```

```

if r < 0 or r >= rows or c < 0 or c >= cols:
    return
char = board[r][c]
if char not in node.children:
    return

board[r][c] = '#' # Mark visited
next_node = node.children[char]
for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
    dfs(r+dr, c+dc, next_node, path + char)
board[r][c] = char # Unmark

for r in range(rows):
    for c in range(cols):
        dfs(r, c, trie.root, "")

return list(result)

```

## 13.2 Segment Tree (Range Queries)

**Definition:** A segment tree is a binary tree for efficient **range queries** (sum, min, max) and **point/range updates** on an array.

### Real-Life Analogy

**Corporate hierarchy reporting:** Instead of asking every employee for their sales ( $O(n)$ ), managers aggregate their team's total. The CEO gets the company total instantly from a few division heads.

```

class SegmentTree:
    def __init__(self, nums):
        self.n = len(nums)
        self.tree = [0] * (4 * self.n)
        self._build(nums, 0, 0, self.n - 1)

    def _build(self, nums, node, start, end):
        if start == end:
            self.tree[node] = nums[start]
            return
        mid = (start + end) // 2
        self._build(nums, 2*node+1, start, mid)
        self._build(nums, 2*node+2, mid+1, end)
        self.tree[node] = self.tree[2*node+1] + self.tree[2*node+2]

```



```

def update(self, idx, val, node=0, start=0, end=None):
    """Update single element - O(log n)."""
    if end is None: end = self.n - 1
    if start == end:
        self.tree[node] = val
        return
    mid = (start + end) // 2
    if idx <= mid:
        self.update(idx, val, 2*node+1, start, mid)
    else:
        self.update(idx, val, 2*node+2, mid+1, end)
    self.tree[node] = self.tree[2*node+1] + self.tree[2*node+2]

def query(self, l, r, node=0, start=0, end=None):
    """Range sum query - O(log n)."""
    if end is None: end = self.n - 1
    if r < start or end < l:
        return 0
    if l <= start and end <= r:
        return self.tree[node]
    mid = (start + end) // 2
    return (self.query(l, r, 2*node+1, start, mid) +
            self.query(l, r, 2*node+2, mid+1, end))

# Build: O(n) | Query: O(log n) | Update: O(log n)

```

## 13.3 Bit Manipulation

### Essential Bit Operations

```

# Basic operations
a & b    # AND - both bits 1
a | b    # OR - either bit 1
a ^ b    # XOR - bits differ
~a       # NOT - flip all bits
a << n   # Left shift - multiply by 2^n
a >> n   # Right shift - divide by 2^n

# Useful tricks
n & 1    # Check if odd
n & (n - 1) # Remove lowest set bit
n & (-n)  # Isolate lowest set bit
bin(n).count('1') # Count set bits (popcount)

```

```

# Check if power of 2
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

# Single Number – find the element that appears once (others twice)
def single_number(nums):
    result = 0
    for num in nums:
        result ^= num    # XOR cancels pairs
    return result

# Time: O(n) | Space: O(1)

# Count set bits
def count_bits(n):
    count = 0
    while n:
        n &= (n - 1)    # Remove lowest set bit
        count += 1
    return count

# Time: O(k) where k = number of set bits

```

## 13.4 Sliding Window Advanced

### Minimum Window Substring (LC #76)

```

from collections import Counter

def min_window(s, t):
    """Smallest substring of s containing all chars of t."""
    if not t or not s:
        return ""

    need = Counter(t)
    have = {}
    have_count, need_count = 0, len(need)
    result = ""
    result_len = float('inf')
    left = 0

    for right in range(len(s)):
        char = s[right]
        have[char] = have.get(char, 0) + 1

        if char in need and have[char] == need[char]:
            need_count -= 1

            if need_count == 0:
                if result_len > right - left + 1:
                    result = s[left:right+1]
                    result_len = right - left + 1
                left += 1

```

```

        have_count += 1

    while have_count == need_count:
        # Update result
        window_len = right - left + 1
        if window_len < result_len:
            result_len = window_len
            result = s[left:right + 1]

        # Shrink window
        left_char = s[left]
        have[left_char] -= 1
        if left_char in need and have[left_char] < need[left_char]:
            have_count -= 1
        left += 1

    return result

# Time: O(|s| + |t|) | Space: O(|s| + |t|)

```

## 13.5 Pattern Recognition Tips

-  “Autocomplete / prefix search” → Trie
-  “Word dictionary with wildcards” → Trie + DFS
-  “Range sum/min/max with updates” → Segment Tree
-  “Find element appearing once” → XOR bit manipulation
-  “Count set bits / power of 2” → Bit tricks
-  “Minimum window containing all chars” → Sliding Window + HashMap

## 13.6 Common Mistakes

-  Using Trie when HashMap suffices — Trie is for prefix operations
-  Segment tree off-by-one — 0-indexed vs 1-indexed
-  Forgetting XOR properties —  $a \oplus a = 0$ ,  $a \oplus 0 = a$
-  Not considering negative numbers in bit manipulation

## 13.7 Practice Problems

| # | Problem                  | Structure | Difficulty |
|---|--------------------------|-----------|------------|
| 1 | Implement Trie (LC #208) | Trie      | Medium     |

| # | Problem                           | Structure           | Difficulty |
|---|-----------------------------------|---------------------|------------|
| 2 | Single Number (LC #136)           | Bit XOR             | Easy       |
| 3 | Word Search II (LC #212)          | Trie + Backtracking | Hard       |
| 4 | Range Sum Query Mutable (LC #307) | Segment Tree        | Medium     |
| 5 | Minimum Window Substring (LC #76) | Sliding Window      | Hard       |



## Revision Summary

- ✓ Trie:  $O(L)$  insert/search, perfect for prefix problems
- ✓ Segment Tree:  $O(\log n)$  range queries and updates
- ✓ XOR:  $a \oplus a = 0$  – finds single element in  $O(n)$ ,  $O(1)$  space
- ✓ Bit tricks:  $n \& (n-1)$  removes lowest set bit
- ✓ Advanced sliding window: expand right, shrink left
- ✓ Use the right data structure for the right problem

*Next: Module 14 — FAANG Interview Cheatsheet →*

# Module 14: FAANG Interview Cheatsheet

## Master Pattern Map — “What Technique Do I Use?”

| PROBLEM TYPE → TECHNIQUE             |                                 |
|--------------------------------------|---------------------------------|
| "Find pair summing to X"<br>(sorted) | → HashMap or Two Pointers       |
| "Contiguous subarray"                | → Sliding Window / Prefix Sum   |
| "Maximum subarray"                   | → Kadane's Algorithm            |
| "Subarray sum equals K"              | → Prefix Sum + HashMap          |
| "Kth largest/smallest"               | → Heap (size k)                 |
| "Top K frequent"                     | → Counter + Heap or Bucket Sort |
| "Sorted array search"                | → Binary Search                 |
| "Min/max valid value"                | → Binary Search on Answer       |
| "Next greater/smaller"               | → Monotonic Stack               |
| "Matching/balancing"                 | → Stack                         |
| "Level-by-level tree"                | → BFS (Queue)                   |
| "Tree path/depth"                    | → DFS (Recursion)               |
| "Shortest path (unweighted)"         | → BFS                           |
| "Shortest path (weighted)"           | → Dijkstra (Heap)               |
| "Connected components"               | → DFS/BFS or Union-Find         |
| "Task ordering"                      | → Topological Sort              |
| "Generate all subsets"               | → Backtracking                  |
| "Generate all permutations"          | → Backtracking                  |

|                                 |                                     |
|---------------------------------|-------------------------------------|
| "Number of ways"                | → DP                                |
| "Min/max cost"                  | → DP or Greedy                      |
| "Interval scheduling"           | → Sort by end + Greedy              |
| "Prefix/autocomplete"           | → Trie                              |
| "Find single/unique element"    | → XOR (Bit Manipulation)            |
| "Cycle detection (Linked List)" | → Fast/Slow Pointers                |
| "Merge K sorted"                | → Min-Heap                          |
| "Running median"                | → Two Heaps                         |
| "LRU Cache"                     | → HashMap + Doubly Linked List      |
| "Palindrome"                    | → Two Pointers / Expand from Center |
| "Anagram"                       | → Character Frequency (Counter)     |

## Complexity Quick Reference

### Data Structures

| Structure      | Access | Search      | Insert      | Delete      | Space           |
|----------------|--------|-------------|-------------|-------------|-----------------|
| Array          | $O(1)$ | $O(n)$      | $O(n)$      | $O(n)$      | $O(n)$          |
| Linked List    | $O(n)$ | $O(n)$      | $O(1)^*$    | $O(1)^*$    | $O(n)$          |
| Hash Table     | —      | $O(1)$ avg  | $O(1)$ avg  | $O(1)$ avg  | $O(n)$          |
| BST (balanced) | —      | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | $O(n)$          |
| Heap           | —      | $O(n)$      | $O(\log n)$ | $O(\log n)$ | $O(n)$          |
| Trie           | —      | $O(L)$      | $O(L)$      | $O(L)$      | $O(N \times L)$ |
| Stack/Queue    | —      | —           | $O(1)$      | $O(1)$      | $O(n)$          |

\*If node reference is known

### Sorting Algorithms

| Algorithm  | Best          | Average       | Worst         | Space  | Stable |
|------------|---------------|---------------|---------------|--------|--------|
| Merge Sort | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$ | ✓      |

| Algorithm     | Best          | Average       | Worst         | Space       | Stable |
|---------------|---------------|---------------|---------------|-------------|--------|
| Quick Sort    | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | ✗      |
| Heap Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$      | ✗      |
| Timsort       | $O(n)$        | $O(n \log n)$ | $O(n \log n)$ | $O(n)$      | ✓      |
| Counting Sort | $O(n+k)$      | $O(n+k)$      | $O(n+k)$      | $O(k)$      | ✓      |

## Graph Algorithms

| Algorithm        | Time             | Space  | Use Case                      |
|------------------|------------------|--------|-------------------------------|
| BFS              | $O(V+E)$         | $O(V)$ | Shortest path (unweighted)    |
| DFS              | $O(V+E)$         | $O(V)$ | Path finding, cycle detection |
| Dijkstra         | $O((V+E)\log V)$ | $O(V)$ | Shortest path (weighted)      |
| Bellman-Ford     | $O(V \times E)$  | $O(V)$ | Negative weights              |
| Topological Sort | $O(V+E)$         | $O(V)$ | DAG ordering                  |
| Union-Find       | $O(\alpha(n))$   | $O(n)$ | Connected components          |

## Top 50 Must-Solve Problems (By Category)

### Arrays & Strings (10)

1. Two Sum (#1) ★
2. Best Time to Buy & Sell Stock (#121) ★
3. Contains Duplicate (#217)
4. Product of Array Except Self (#238)
5. Maximum Subarray (#53)
6. 3Sum (#15)
7. Container With Most Water (#11)
8. Longest Substring Without Repeating Characters (#3)
9. Group Anagrams (#49)
10. Trapping Rain Water (#42) 🔥

### Linked Lists (5)

11. Reverse Linked List (#206) ★
12. Merge Two Sorted Lists (#21)
13. Linked List Cycle (#141)
14. Remove Nth From End (#19)
15. LRU Cache (#146) 🔥

## Stacks & Queues (4)

- 16. Valid Parentheses (#20) ★
- 17. Min Stack (#155)
- 18. Daily Temperatures (#739)
- 19. Largest Rectangle in Histogram (#84) 🔥

## Trees (7)

- 20. Maximum Depth (#104) ★
- 21. Invert Binary Tree (#226)
- 22. Validate BST (#98)
- 23. Level Order Traversal (#102)
- 24. Lowest Common Ancestor (#236)
- 25. Binary Tree Maximum Path Sum (#124) 🔥
- 26. Serialize and Deserialize (#297) 🔥

## Heaps (3)

- 27. Kth Largest Element (#215)
- 28. Merge K Sorted Lists (#23) 🔥
- 29. Find Median from Data Stream (#295) 🔥

## Graphs (6)

- 30. Number of Islands (#200) ★
- 31. Clone Graph (#133)
- 32. Course Schedule (#207)
- 33. Pacific Atlantic Water Flow (#417)
- 34. Word Ladder (#127)
- 35. Alien Dictionary (#269) 🔥

## Dynamic Programming (8)

- 36. Climbing Stairs (#70) ★
- 37. House Robber (#198)
- 38. Coin Change (#322)
- 39. Longest Increasing Subsequence (#300)
- 40. Longest Common Subsequence (#1143)
- 41. Word Break (#139)
- 42. Edit Distance (#72)
- 43. Decode Ways (#91)



## Backtracking (4)

- 44. Subsets (#78)
- 45. Permutations (#46)
- 46. Combination Sum (#39)
- 47. N-Queens (#51) 🔥

## Binary Search (3)

- 48. Search in Rotated Sorted Array (#33)
- 49. Find Minimum in Rotated Sorted Array (#153)
- 50. Median of Two Sorted Arrays (#4) 🔥

★ = Must-know basics | 🔥 = Hard but frequently asked at FAANG

---



## Interview Execution Framework

### The 5-Step Process (Use for EVERY problem)

1. UNDERSTAND (2 min)
  - Repeat the problem back
  - Ask clarifying questions
  - Identify input/output types and constraints
  - Walk through 1-2 examples
2. PLAN (5 min)
  - Identify the pattern/technique
  - Start with brute force
  - Optimize: "Can I use a hashmap? Two pointers? Sorting?"
  - State time/space complexity of your approach
  - Get interviewer buy-in before coding
3. CODE (15 min)
  - Write clean, readable code
  - Use meaningful variable names
  - Talk through your code as you write
  - Handle edge cases
4. TEST (3 min)
  - Trace through with the given example
  - Test edge cases: empty input, single element, large input
  - Fix bugs calmly
5. OPTIMIZE (if time permits)
  - Can you reduce space?

- Can you eliminate a loop?
- Discuss tradeoffs

## Clarifying Questions to Ask

Arrays: "Is the array sorted? Can there be duplicates? Negative numbers?"

Strings: "ASCII or Unicode? Case-sensitive? Can it be empty?"

Trees: "Binary tree or BST? Can it have duplicates? Is it balanced?"

Graphs: "Directed or undirected? Weighted? Can there be cycles?"

General: "What are the constraints on n? Can I modify the input?"

## Red Flags to Avoid

- ✗ Jumping straight into code without a plan
  - ✗ Going silent for more than 30 seconds
  - ✗ Not testing your code
  - ✗ Getting flustered by hints – take them gracefully
  - ✗ Saying "I've seen this problem before" – solve it fresh
  - ✗ Writing messy code with single-letter variables
- 



## Python Tips for Interviews

```
# Essential imports
from collections import defaultdict, Counter, deque, OrderedDict
from heapq import heappush, heappop, heapify
from bisect import bisect_left, bisect_right, insort
from functools import lru_cache
from itertools import combinations, permutations
import math

# Infinity
float('inf')          # Positive infinity
float('-inf')         # Negative infinity

# Integer bounds
import sys
sys.maxsize           # Max integer

# Useful one-liners
max(arr)              # Max element
min(arr)              # Min element
sum(arr)              # Sum
sorted(arr)           # Sorted copy
```

```
enumerate(arr)      # (index, value) pairs
zip(a, b)           # Pair elements
any(condition)      # True if any condition met
all(condition)      # True if all conditions met

# List comprehensions
[x**2 for x in range(10)]          # Squares
[x for x in arr if x > 0]          # Filter
[[0]*cols for _ in range(rows)]   # 2D array

# String tricks
s.isalpha()          # Only letters?
s.isdigit()          # Only digits?
s.isalnum()          # Letters or digits?
ord('a')             # 97 (ASCII value)
chr(97)              # 'a'
```

---

## Final Study Plan (4-Week Sprint)

Week 1: Arrays, Strings, Hashing, Two Pointers

- 3 Easy + 2 Medium per day
- Master: Two Sum, Valid Anagram, Sliding Window

Week 2: Linked Lists, Stacks, Queues, Binary Search

- 2 Medium + 1 Hard per day
- Master: Reverse LL, Valid Parentheses, Search Rotated

Week 3: Trees, Heaps, Graphs

- 2 Medium + 1 Hard per day
- Master: BFS/DFS, Dijkstra, Topological Sort

Week 4: DP, Greedy, Backtracking, Mock Interviews

- Focus on patterns, not problem count
- Do 2-3 timed mock interviews
- Review all revision summaries

---

## Golden Rules

1. **Understand before you code** — 5 min thinking saves 20 min debugging
2. **Start brute force, then optimize** — never jump to the optimal solution
3. **Communicate constantly** — narrate your thought process
4. **Handle edge cases** — empty input, single element, negative numbers
5. **Know your complexities** — state time AND space for every solution

6. **Practice under time pressure** — 30 min per medium, 45 min per hard
  7. **Review your mistakes** — a mistake solved twice is a pattern learned
  8. **Sleep well before the interview** — a rested mind > 100 more problems
- 

🎓 *Congratulations! You've completed the entire DSA course. Now go ace those interviews!*

---